

```
In [1]: import numpy as np
import pandas as pd
import itertools
import math

import scipy
from scipy.signal import hilbert, find_peaks, peak_prominences, butter, filtfilt
import statsmodels.api as sm

import matplotlib.pyplot as plt

from parselmouth.praat import call
from parselmouth import Sound
```

Goal

The goal of this notebook is to establish clear definitions and visualizations for the following vibrato measures:

- **Vibrato Rate (FM)**
- **Vibrato Extent (FM)**
- **Vibrato Rate (AM)**
- **Vibrato Extent (AM)**
- **Phase Difference (AM–FM)**

Synthetic signals are used to provide known ground truth values for all parameters.

```
In [2]: import numpy as np

def generate_vibrato_signal(
    f0=100.0,
    fs=10000,
    T=4.0,
    VibRate_FM=6.0,
    VibExtent_cents=40.0,
    baseAmp=1,
    VibRate_AM=10.0,
    AM_depth=0.1, # fraction 0 → 1
    fm_phase = 0,
    am_phase = 0
):
    """
    Vibrato signal with controllable AM depth.

    AM_depth = 1.0 → full modulation (0.5*baseAmp → 1.5*baseAmp)
    AM_depth = 0.2 → 20% modulation (0.9*baseAmp → 1.1*baseAmp)
    """
    dt = 1.0 / fs
    t = np.arange(0, T + dt, dt)

    # Frequency modulation (Linear Hz)
    cents_deviation = VibExtent_cents * np.sin(2.0 * np.pi * VibRate_FM * t + fm_phase)
    VibSignalFreq = f0 * (2.0 ** (cents_deviation / 1200.0))
    VibSignalPhase = 2.0 * np.pi * np.cumsum(VibSignalFreq) / fs

    # Amplitude envelope with controlled depth
    VibAmpInst = baseAmp * (1 + AM_depth * np.cos(2.0 * np.pi * VibRate_AM * t + am_phase))
```

```

# Output
VibratoSignal = VibAmpInst * np.sin(VibSignalPhase)

return VibratoSignal, t

```

Example Signal from Parameter Sweep

This section visualizes a representative synthetic vibrato signal with simultaneous FM and AM, using parameters drawn from the sweep ranges.

```

In [3]: # --- Characteristic sweep values ---
f0 = 294.0          # Hz
fs = 10000          # Hz
T = 2.0             # seconds

vibRate_fm = 6.0     # Hz
vibExtent_cents = 40.0 # cents

vibRate_am = 6.0     # Hz
am_depth = 0.02      # 1% AM depth

fm_phase = 0.0
am_phase = 0.0#np.pi / 2

```

```

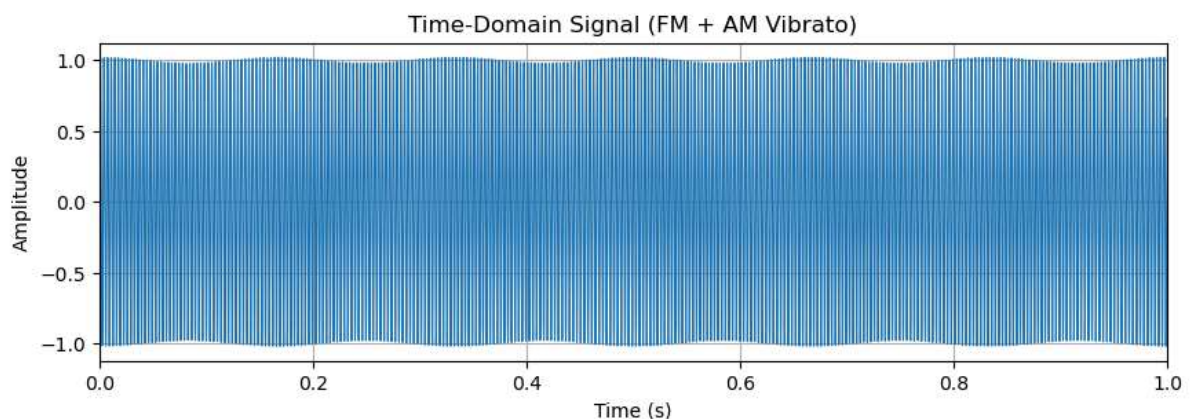
In [5]: signal, t = generate_vibrato_signal(
    f0=f0,
    fs=fs,
    T=T,
    VibRate_FM=vibRate_fm,
    VibExtent_cents=vibExtent_cents,
    baseAmp=1.0,
    VibRate_AM=vibRate_am,
    AM_depth=am_depth,
    fm_phase=fm_phase,
    am_phase=am_phase
)

```

```

In [6]: plt.figure(figsize=(10, 3))
plt.plot(t, signal, linewidth=0.8)
plt.title("Time-Domain Signal (FM + AM Vibrato)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.xlim(0, 1) # zoom in
plt.grid(True)
plt.show()

```



```
In [7]: def bandpass_filter(signal, fs, f_center, bandwidth=1.0, order=4):
    nyquist = fs / 2
    low = (f_center - bandwidth/2) / nyquist
    high = (f_center + bandwidth/2) / nyquist
    if low <= 0:
        low = 1e-6 # just above 0
    if high >= 1:
        high = 0.999999
    if low >= high:
        raise ValueError(
            f"Invalid bandpass: f_center={f_center}, bandwidth={bandwidth}, fs={fs}"
            f"-> low={low:.3f}, high={high:.3f}"
        )

    b, a = butter(order, [low, high], btype='band')
    return filtfilt(b, a, signal)

def returnContour(data, samplerate):
    ###CALCULATE vibRate
    sound = Sound(data, samplerate)
    #Creates PRAAT sound file from .wav array, default 44100 Hz sampling frequency:
    # pitch = call(sound, "To Pitch", 0.001, 60, 1000) #Use time steps of 0.001 for
    pitch = call(sound, "To Pitch", 0.001, 260, 1000) # Raising the low_frequency v
    pitch_contour = pitch.selected_array['frequency']

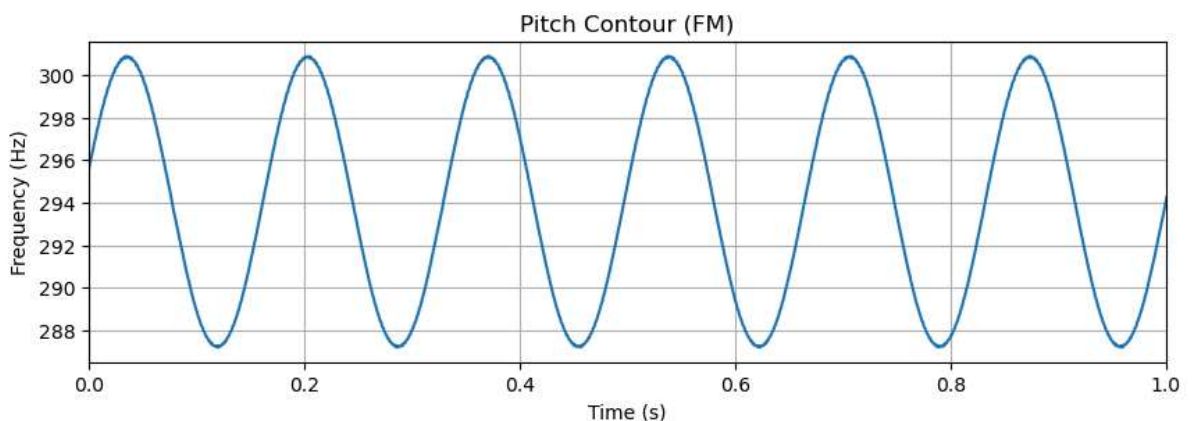
    f_s_contour = pitch.selected_array['frequency'].size/sound.values.size*sound.sa
    return pitch_contour, f_s_contour

def returnEnvelope(data, samplerate, meanFreq):
    bandwidth_h = meanFreq#2 * f_h * (2**(vib_cents/1200) - 1) * 1.3
    filtered = bandpass_filter(data, samplerate, meanFreq, bandwidth_h)
    env_pa = np.abs(hilbert(filtered))
    return env_pa, filtered, samplerate
```

```
In [8]: pitch_contour, f_s_contour = returnContour(signal, fs)
envelope, filtered, _ = returnEnvelope(signal, fs, f0)
```

```
In [9]: t_pitch = np.arange(len(pitch_contour)) / f_s_contour
```

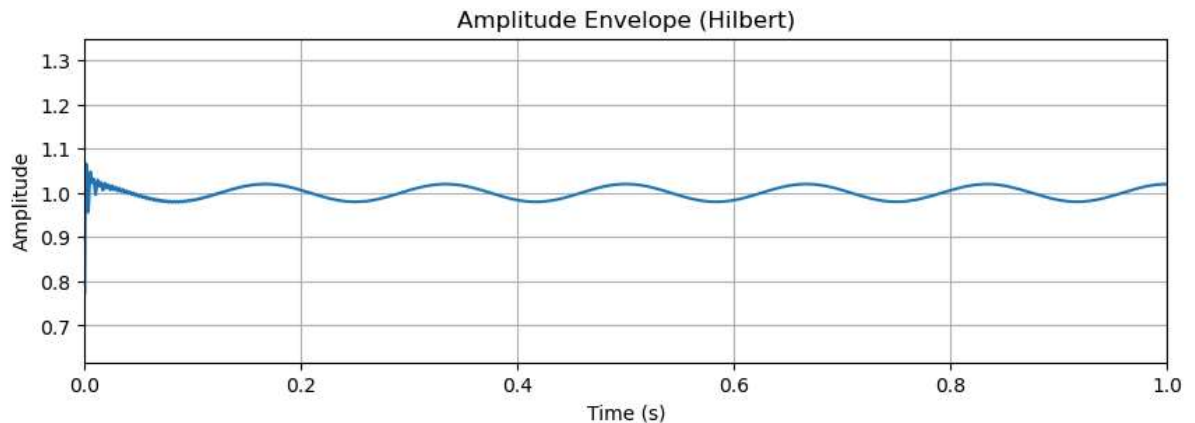
```
plt.figure(figsize=(10, 3))
plt.plot(t_pitch, pitch_contour)
plt.title("Pitch Contour (FM)")
plt.xlabel("Time (s)")
plt.ylabel("Frequency (Hz)")
plt.xlim(0,1)
plt.grid(True)
plt.show()
```



So the pitch contour shows 6 oscillations in one second. So far so good.

```
In [10]: t_env = np.arange(len(envelope)) / fs

plt.figure(figsize=(10, 3))
plt.plot(t_env, envelope)
plt.title("Amplitude Envelope (Hilbert)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.xlim(0, 1)
plt.grid(True)
plt.show()
```



The amplitude contour likewise shows 6 oscillations in one second with some noise at the boundary of the signal.

```
In [14]: def compute_vibrato_extent_cents(pitch_contour, f_s_contour, vibFreq, windowFactor=
if np.isnan(vibFreq):
    #vibFreq = 5.5
    ampCents = np.nan
    #print(str(ampCents))
    return ampCents
wavelength = 1.0/vibFreq*f_s_contour # in frames
# print('wavelength: ', str(wavelength))
try:
    window = math.floor(wavelength*windowFactor)
except ValueError:
    window = math.floor(1.0/5.5*f_s_contour*0.75)
#
if window == 0:
    window = 1
# print('Window: ', str(window))
maxPeaks = find_peaks(pitch_contour, distance=window)[0]
prominences = scipy.signal.peak_prominences(pitch_contour, maxPeaks)[0]/2

ampEstimate = prominences[1:-1].mean()
meanFreq = pitch_contour[1:-1].mean()
# ampStd = prominences[1:-2].std()
ampStd = prominences[1:-1].std()
#
vibExtent_cents = 1200*np.log(1 + ampEstimate/meanFreq)/np.log(2)
return vibExtent_cents, maxPeaks, prominences#, vibStd # amplitude in cents
```

```
In [15]: vibExtent_cents, peaks, half_prominences = compute_vibrato_extent_cents(pitch_contour)
print(f"Estimated vibrato extent: {vibExtent_cents:.4f} (cents)")
# --- Visualization ---
plt.figure(figsize=(12,4))
```

```

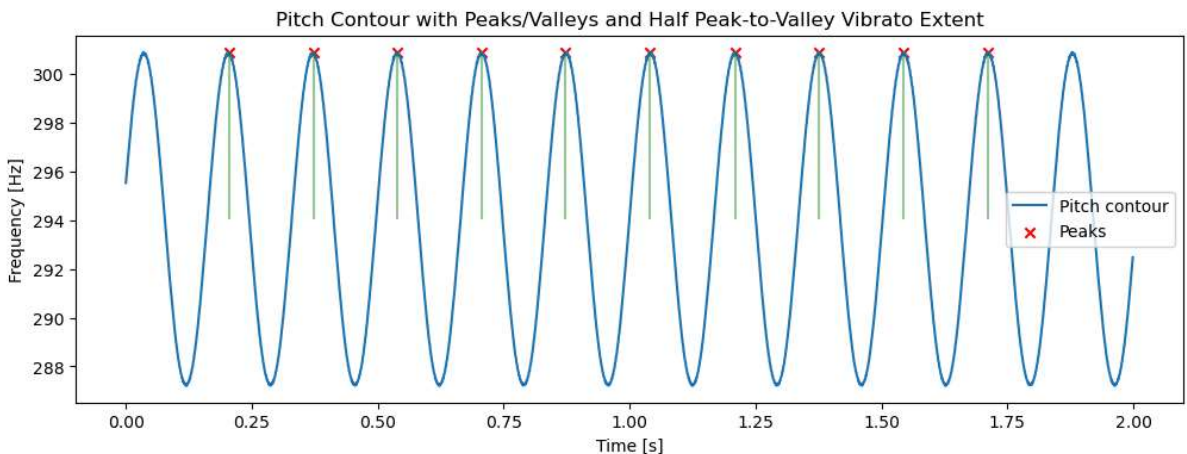
plt.plot(t_pitch, pitch_contour, label='Pitch contour')
plt.scatter(t_pitch[peaks[1:-1]], pitch_contour[peaks[1:-1]], color='red', marker='x')
# plt.scatter(t_pitch[valleys], pitch_contour[valleys], color='green', marker='o', label='Valleys')

# Draw half peak-to-valley lines
for p, prom in zip(peaks[1:-1], half_prominences[1:-1]):
    plt.vlines(t_pitch[p], pitch_contour[p]-prom, pitch_contour[p], color='green', label='Half peak-to-valley lines')

plt.xlabel('Time [s]')
plt.ylabel('Frequency [Hz]')
plt.title('Pitch Contour with Peaks/Valleys and Half Peak-to-Valley Vibrato Extent')
plt.legend()
plt.show()

```

Estimated vibrato extent: 39.8211 (cents)



```

In [16]: # --- Function to estimate vibrato extent (AM) ---
def compute_vibrato_extent_hilbert(envelope, samplerate, vibAmpGuess):
    wavelength = 1.0 / vibAmpGuess * samplerate
    window = int(np.floor(wavelength * 0.75))
    if window <= 0:
        window = 1

    maxPeaks = find_peaks(envelope[window:-window], distance=window)[0]
    prominences = peak_prominences(envelope[window:-window], maxPeaks)[0]
    extentEstimate = np.nanmedian(prominences) / 2
    return extentEstimate, (maxPeaks + window), prominences

```

```

In [17]: # --- Estimate vibrato extent ---
extentEstimate, peaks, prominences = compute_vibrato_extent_hilbert(envelope, fs, vibAmpGuess)
half_prominences = prominences / 2
print(f"Estimated vibrato extent: {extentEstimate:.4f} (percent of mean signal amplitude)")

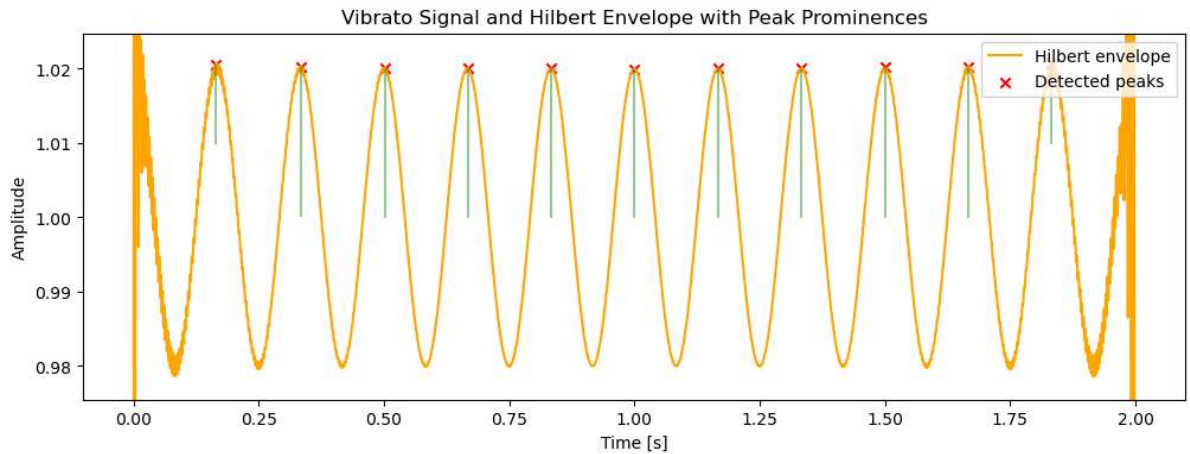
# --- Visualization ---
plt.figure(figsize=(12,4))
# plt.plot(t, signal, label='Vibrato signal')
plt.plot(t, envelope, color='orange', label='Hilbert envelope')
plt.scatter(t[peaks], envelope[peaks], color='red', marker='x', label='Detected peaks')
for p, prom in zip(peaks, half_prominences):
    plt.vlines(t[p], envelope[p]-prom, envelope[p], color='green', alpha=0.4)

yRange = envelope[peaks[1]:peaks[-1]].max()
peak_vals = envelope[peaks[1]:peaks[-1]]
ymin, ymax = peak_vals.min(), peak_vals.max()
yrange = ymax - ymin
plt.ylim(ymin - 0.1*yrange, ymax + 0.1*yrange)
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.title('Vibrato Signal and Hilbert Envelope with Peak Prominences')

```

```
plt.legend()
plt.show()
```

Estimated vibrato extent: 0.0201 (percent of mean signal amplitude)



```
In [18]: def extract_phase_difference(
    envelope,
    pitch_contour,
    fs_env,
    fs_pitch,
    vibRate_f0,
    vibRate_amp,
    rate_tol=0.5 # Hz tolerance
):
    """
    Estimate AM-FM phase difference at the vibrato rate.

    Returns np.nan if vibrato rates do not match within tolerance.
    """

    # --- 1. Reject mismatched rates ---
    if np.isnan(vibRate_f0) or np.isnan(vibRate_amp):
        return np.nan

    if abs(vibRate_f0 - vibRate_amp) > rate_tol:
        return np.nan

    vibRate = 0.5 * (vibRate_f0 + vibRate_amp)

    # --- 2. Resample signals to common length ---
    N = min(len(envelope), len(pitch_contour))
    envelope_rs = resample(envelope, N)
    pitch_rs = resample(pitch_contour, N)

    fs = min(fs_env, fs_pitch)
    t = np.arange(N) / fs

    # --- 3. Remove DC ---
    env = envelope_rs - np.mean(envelope_rs)
    pitch = pitch_rs - np.mean(pitch_rs)

    # --- 4. Reference sinusoids ---
    omega = 2 * np.pi * vibRate
    sin_ref = np.sin(omega * t)
    cos_ref = np.cos(omega * t)

    # --- 5. Phase via projection ---
    def estimate_phase(x):
        a = np.dot(x, cos_ref)
        b = np.dot(x, sin_ref)
```

```

        return np.arctan2(b, a)

    phi_env = estimate_phase(env)
    phi_pitch = estimate_phase(pitch)

    # --- 6. Phase difference ---
    phase_diff = phi_env - phi_pitch
    phase_diff = np.arctan2(np.sin(phase_diff), np.cos(phase_diff))

    return phase_diff

```

```

In [19]: import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import resample

def visualize_phase_difference(
    envelope,
    pitch_contour,
    fs_env,
    fs_pitch,
    vibRate_f0,
    vibRate_amp,
    rate_tol=0.5
):
    phase_diff = extract_phase_difference(
        envelope,
        pitch_contour,
        fs_env,
        fs_pitch,
        vibRate_f0,
        vibRate_amp,
        rate_tol
    )

    if np.isnan(phase_diff):
        print("Phase difference could not be estimated (rate mismatch or NaN).")
        return np.nan

    vibRate = 0.5 * (vibRate_f0 + vibRate_amp)

    # --- Resample ---
    N = min(len(envelope), len(pitch_contour))
    env_rs = resample(envelope, N)
    pitch_rs = resample(pitch_contour, N)

    fs = min(fs_env, fs_pitch)
    t = np.arange(N) / fs

    # --- Remove DC and Min-Max Norm ---
    env = env_rs - np.mean(env_rs)
    sampleMax = np.abs(env[(len(env)//4):(len(env)*3 // 4)]).max()
    env = env / sampleMax
    pitch = pitch_rs - np.mean(pitch_rs)
    pitch = pitch / np.abs(pitch).max()

    # --- Reference ---
    omega = 2 * np.pi * vibRate
    sin_ref = np.sin(omega * t)
    cos_ref = np.cos(omega * t)

    # --- Phase estimation (same as original) ---
    def estimate_phase(x):
        a = np.dot(x, sin_ref)
        b = np.dot(x, cos_ref)

```



```

        return np.arctan2(b, a)

    phi_env = estimate_phase(env)
    phi_pitch = estimate_phase(pitch)

    # --- Reconstructed sinusoids ---
    env_hat = np.cos(omega * t + phi_env)
    pitch_hat = np.cos(omega * t + phi_pitch)

    # --- Visualization ---
    fig, axs = plt.subplots(
        3, 1,
        figsize=(12, 8),
        sharex=True,
        constrained_layout=True
    )

    # 1. Original signals
    axs[0].plot(t, env, label='Envelope (DC removed)', color='orange')
    axs[0].plot(t, pitch, label='Pitch contour (DC removed)', color='blue', alpha=0.3)
    axs[0].set_title('Normed Mean-Centered AM and FM Signals (Time Domain)')
    axs[0].set_ylabel('Amplitude')
    axs[0].set_ylim(pitch.min(), pitch.max())
    axs[0].legend()
    axs[0].grid(True)

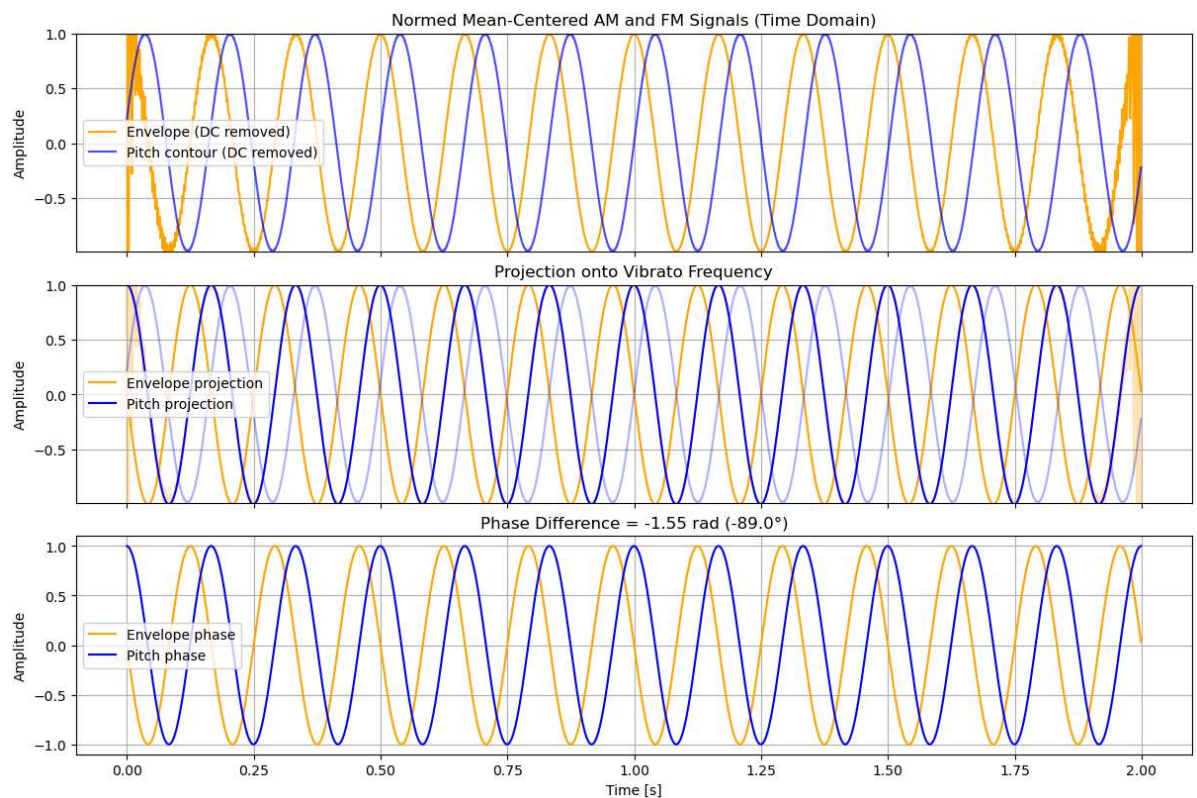
    # 2. Projection onto vibrato sinusoid
    axs[1].plot(t, env, color='orange', alpha=0.3)
    axs[1].plot(t, env_hat, color='orange', label='Envelope projection')
    axs[1].plot(t, pitch, color='blue', alpha=0.3)
    axs[1].plot(t, pitch_hat, color='blue', label='Pitch projection')
    axs[1].set_title('Projection onto Vibrato Frequency')
    axs[1].set_ylabel('Amplitude')
    axs[1].set_ylim(pitch.min(), pitch.max())
    axs[1].legend()
    axs[1].grid(True)

    # 3. Phase difference visualization
    axs[2].plot(t, env_hat, color='orange', label='Envelope phase')
    axs[2].plot(t, pitch_hat, color='blue', label='Pitch phase')
    axs[2].set_title(
        f'Phase Difference = {phase_diff:.2f} rad '
        f'({np.degrees(phase_diff):.1f}°)'
    )
    axs[2].set_xlabel('Time [s]')
    axs[2].set_ylabel('Amplitude')
    axs[2].legend()
    axs[2].grid(True)

    plt.show()

    return phase_diff
phase_diff = visualize_phase_difference(
    envelope,
    pitch_contour,
    fs_env=fs,
    fs_pitch=f_s_contour,
    vibRate_f0=vibRate_fm,
    vibRate_amp=vibRate_am
)

```

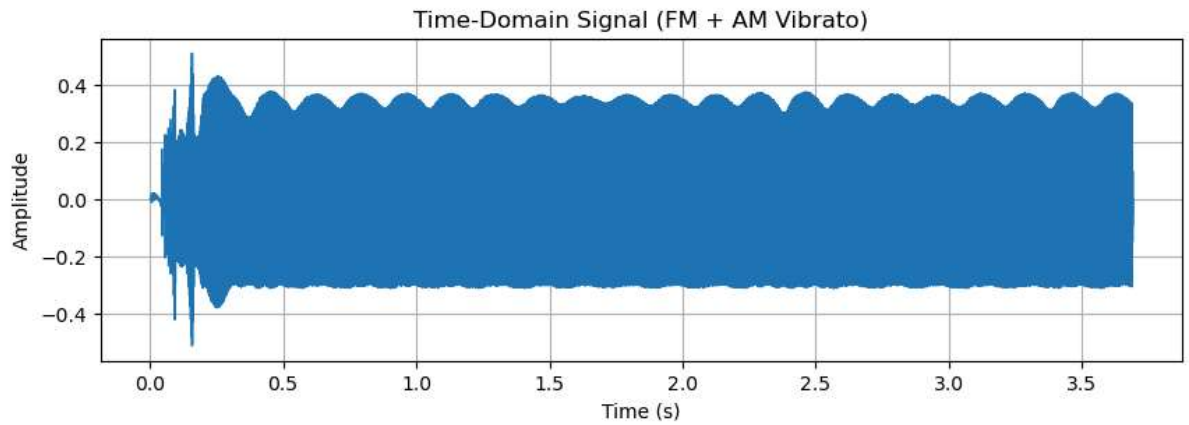



So that makes intuitive sense. The amplitude envelope projections perfectly align because we modeled instantaneous amplitude as cosine. The pitch projection is $\pi/2$ displaced because we modeled it as sine. The two signals are $\pi/2$ phase shifted as neither was phase shifted and one was cosine and the other sine.

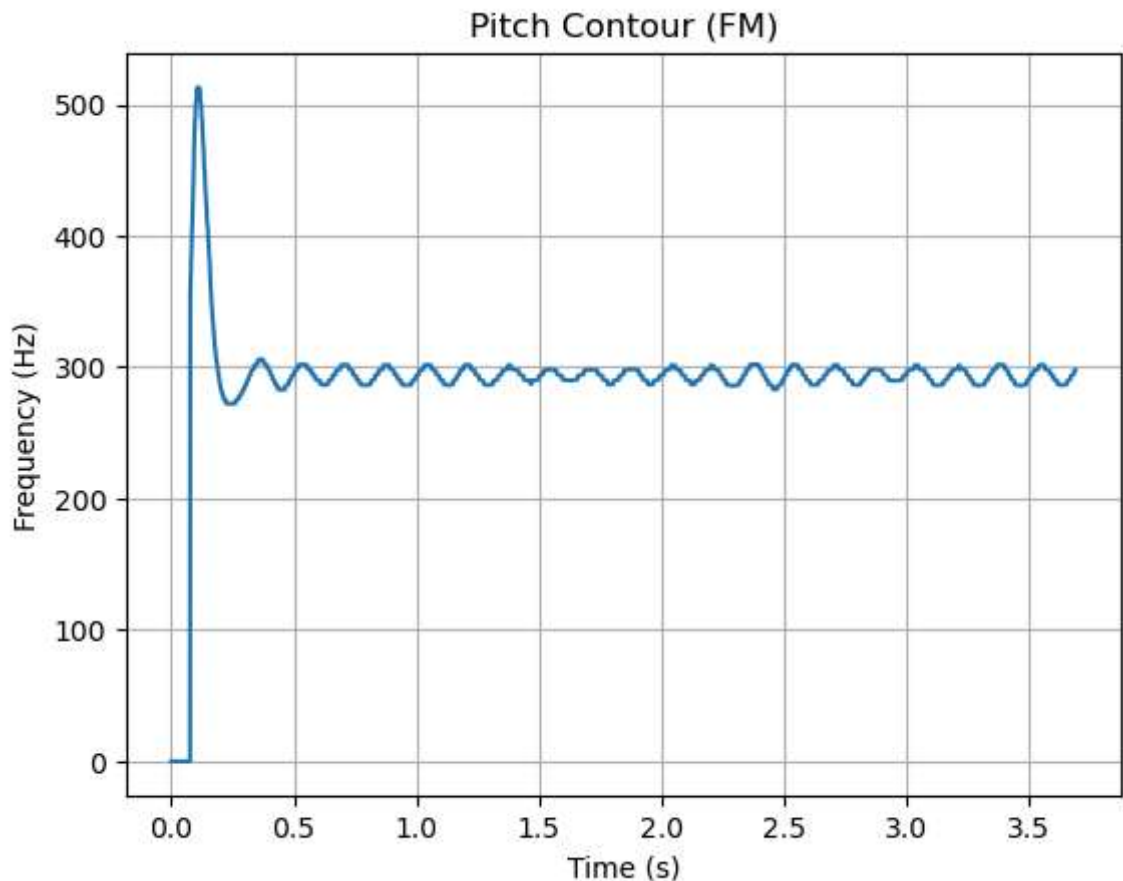
What are the results with a synthesized vocal signal?

```
In [20]: import soundfile as sf
filename = "294_6_40_6_2.wav"
signal, f_s = sf.read(filename)
t = np.arange(0, len(signal))/f_s
f0, vibRateFM_gt, vibExtentFM_gt, Flutter_f0, FlutterDepth = filename.split('.')[0]
FlutterDepthPerc = float(FlutterDepth) / 100
```

```
In [21]: plt.figure(figsize=(10, 3))
plt.plot(t, signal, linewidth=0.8)
plt.title("Time-Domain Signal (FM + AM Vibrato)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
# plt.xlim(0, 1) # zoom in
plt.grid(True)
plt.show()
```



```
In [22]: pitch_contour, f_s_contour = returnContour(signal, f_s)
t_pitch = np.arange(len(pitch_contour)) / f_s_contour
plt.plot(t_pitch, pitch_contour)
plt.title("Pitch Contour (FM)")
plt.xlabel("Time (s)")
plt.ylabel("Frequency (Hz)")
# plt.xlim(0,4)
plt.grid(True)
plt.show()
```



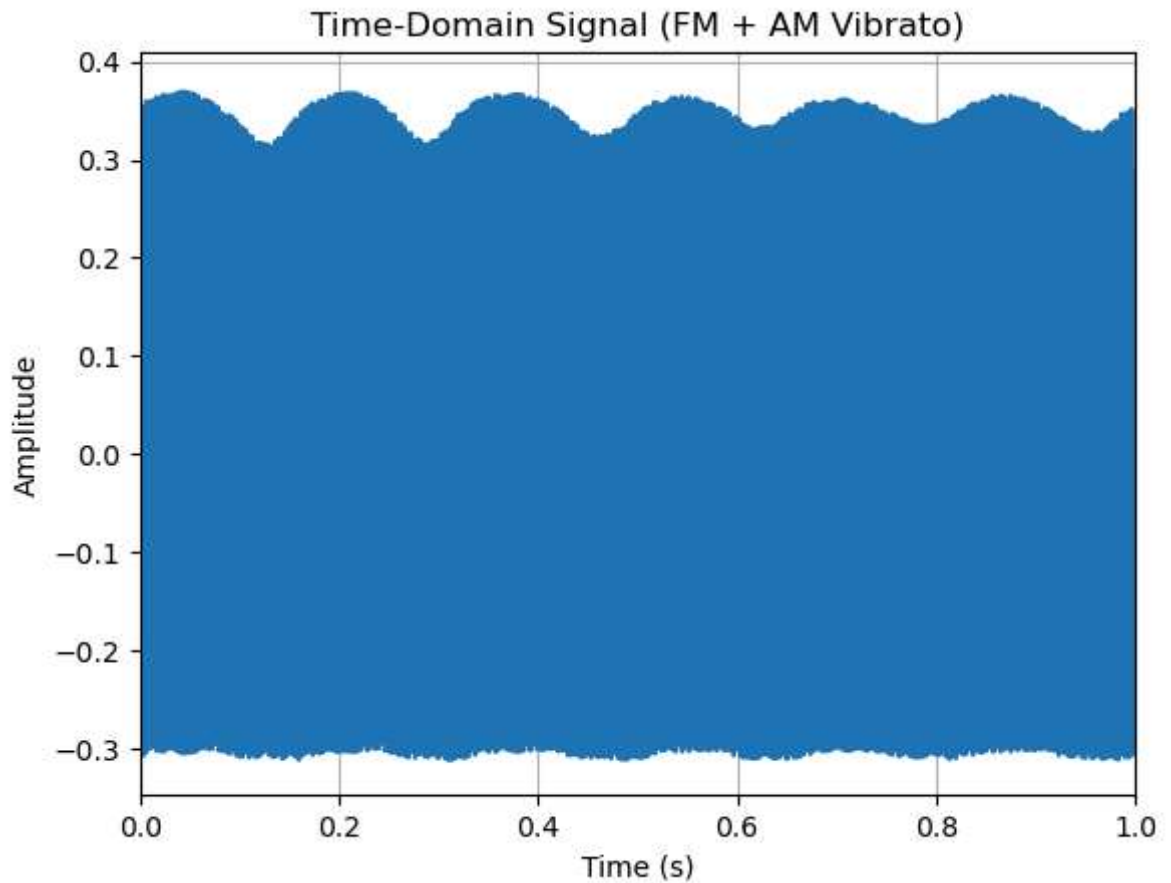
So we have some onset behavior for the synthesized sustained tone. Let's look at the middle 50% of the contour.

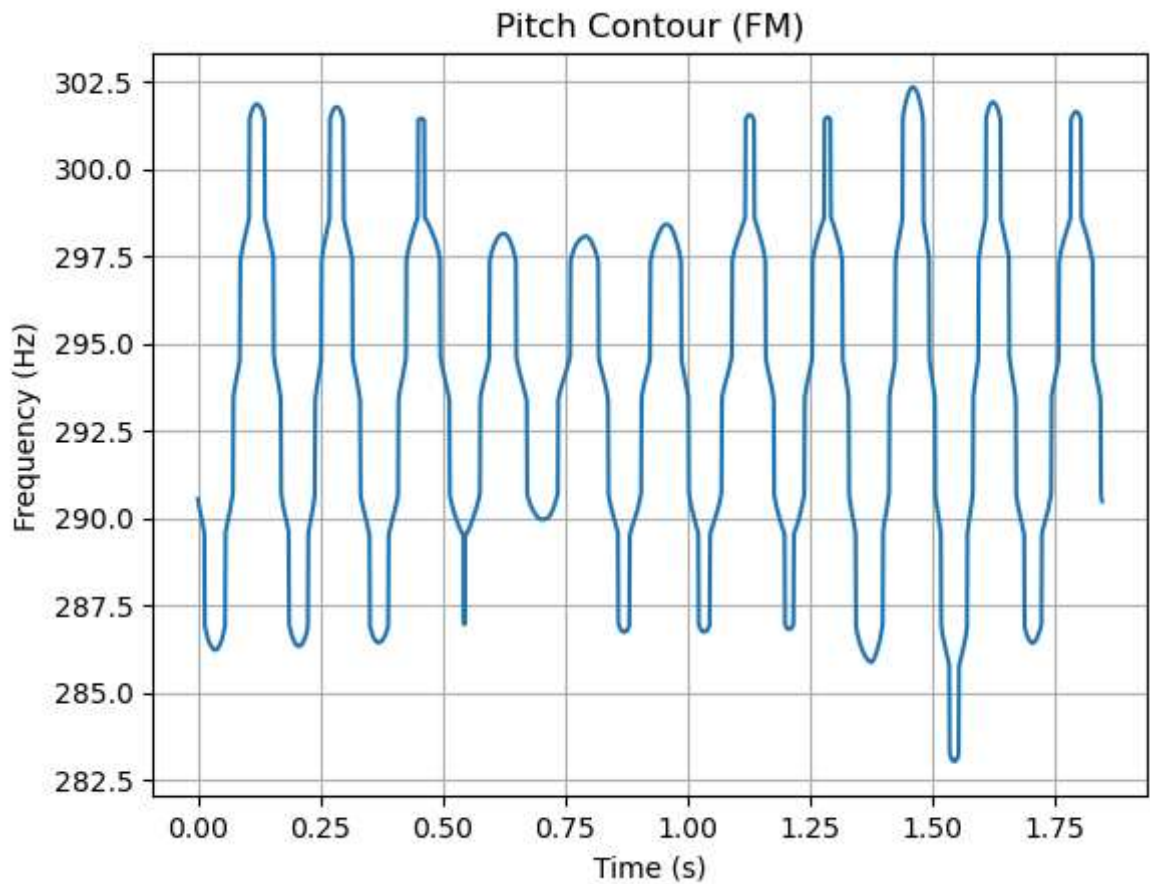
```
In [23]: begin50 = round(len(signal)*.25)
end50 = round(len(signal)*.75)
signal50 = signal[begin50:end50]
t50 = np.arange(0, len(signal50))/f_s
plt.figure()
plt.plot(t50, signal50)
plt.title("Time-Domain Signal (FM + AM Vibrato)")
```

```

plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.xlim(0, 1) # zoom in
plt.grid(True)
plt.show()
pitch_contour, f_s_contour = returnContour(signal[begin50:end50], f_s)
t_pitch = np.arange(len(pitch_contour)) / f_s_contour
plt.figure()
plt.plot(t_pitch, pitch_contour)
plt.title("Pitch Contour (FM)")
plt.xlabel("Time (s)")
plt.ylabel("Frequency (Hz)")
# plt.xlim(0,4)
plt.grid(True)
plt.show()

```





```
In [24]: def autocorrVib3HzLocal(pitch_contour, f_s_contour):
    x = pitch_contour
    n = len(x)
    acorr = sm.tsa.acf(x, nlags=n-1)

    # 95% Confidence interval
    highCI = 1.96 / math.sqrt(n)
    lowCI = -highCI

    # Desired frequency range: 3-10 Hz
    frame10Hz = math.floor(f_s_contour / 10)
    frame3Hz = math.floor(f_s_contour / 3)

    # Find Local maxima in desired range
    acorr_segment = acorr[frame10Hz:frame3Hz]
    peaks, _ = find_peaks(acorr_segment)

    vibratoFreq = np.nan
    if len(peaks) > 0:
        # Get correlation values for each peak
        peak_values = acorr_segment[peaks]

        # Filter only those above the confidence interval
        valid_indices = np.where(peak_values > highCI)[0]

        if len(valid_indices) > 0:
            # Find the *highest* Local maximum
            best_peak_idx = valid_indices[np.argmax(peak_values[valid_indices])]
            maxLag = peaks[best_peak_idx] + frame10Hz
            vibratoFreq = f_s_contour / maxLag
        # else: vibratoFreq remains np.nan

    return vibratoFreq
```

```

In [25]: def visualize_autocorrVib3HzLocal(pitch_contour, f_s_contour):
    x = pitch_contour
    n = len(x)

    # --- Autocorrelation ---
    acorr = sm.tsa.acf(x, nlags=n-1)

    # 95% confidence interval
    highCI = 1.96 / math.sqrt(n)
    lowCI = -highCI

    # Frequency → Lag conversion
    frame10Hz = math.floor(f_s_contour / 10)
    frame3Hz = math.floor(f_s_contour / 3)

    # Segment of interest
    acorr_segment = acorr[frame10Hz:frame3Hz]
    peaks, _ = find_peaks(acorr_segment)

    vibratoFreq = np.nan
    best_lag = None

    if len(peaks) > 0:
        peak_values = acorr_segment[peaks]
        valid_indices = np.where(peak_values > highCI)[0]

        if len(valid_indices) > 0:
            best_peak_idx = valid_indices[np.argmax(peak_values[valid_indices])]
            best_lag = peaks[best_peak_idx] + frame10Hz
            vibratoFreq = f_s_contour / best_lag

    # --- Visualization ---
    lags = np.arange(len(acorr))
    freqs = f_s_contour / np.maximum(lags, 1) # avoid div-by-zero

    plt.figure(figsize=(12, 5))
    plt.plot(freqs, acorr, label='Autocorrelation', color='black')

    # Highlight search band
    plt.axvspan(3, 10, color='gray', alpha=0.2, label='Search range (3–10 Hz)')

    # Confidence interval
    plt.axhline(highCI, color='red', linestyle='--', label='95% CI')
    plt.axhline(lowCI, color='red', linestyle='--')

    # Plot peaks
    if len(peaks) > 0:
        peak_lags = peaks + frame10Hz
        peak_freqs = f_s_contour / peak_lags
        plt.scatter(
            peak_freqs,
            acorr[peak_lags],
            color='blue',
            label='Local maxima'
        )

    # Highlight chosen peak
    if best_lag is not None:
        best_freq = f_s_contour / best_lag
        plt.scatter(
            best_freq,
            acorr[best_lag],
            color='orange',

```

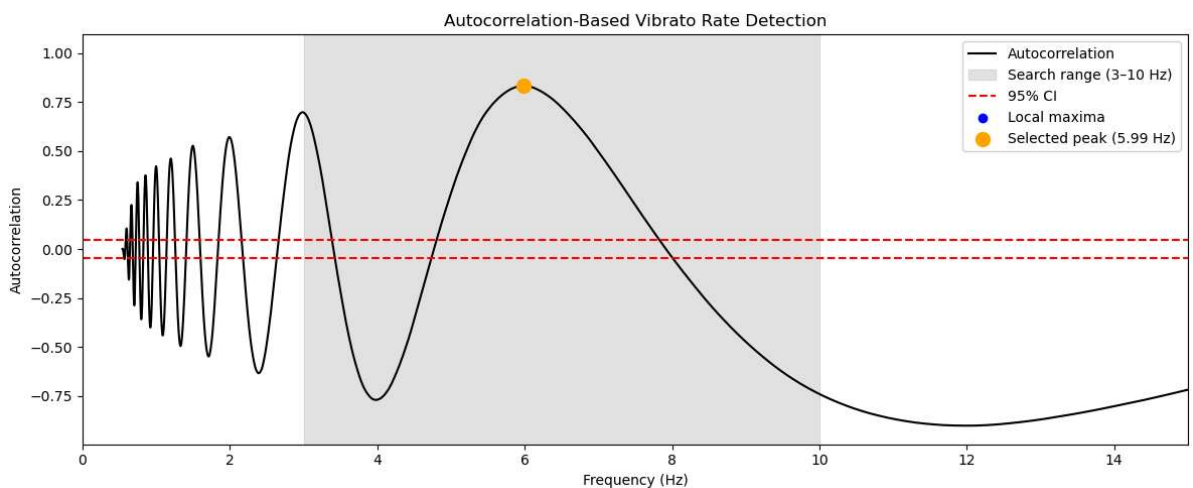
```

        s=100,
        zorder=5,
        label=f'Selected peak ({best_freq:.2f} Hz)'
    )

    plt.xlim(0, 15)
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Autocorrelation')
    plt.title('Autocorrelation-Based Vibrato Rate Detection')
    plt.legend()
    plt.tight_layout()
    plt.show()

    return vibratoFreq
vib_rate_est = visualize_autocorrVib3HzLocal(pitch_contour, f_s_contour)
print(f"Estimated vibrato rate: {vib_rate_est:.2f} Hz")

```



Estimated vibrato rate: 5.99 Hz

```

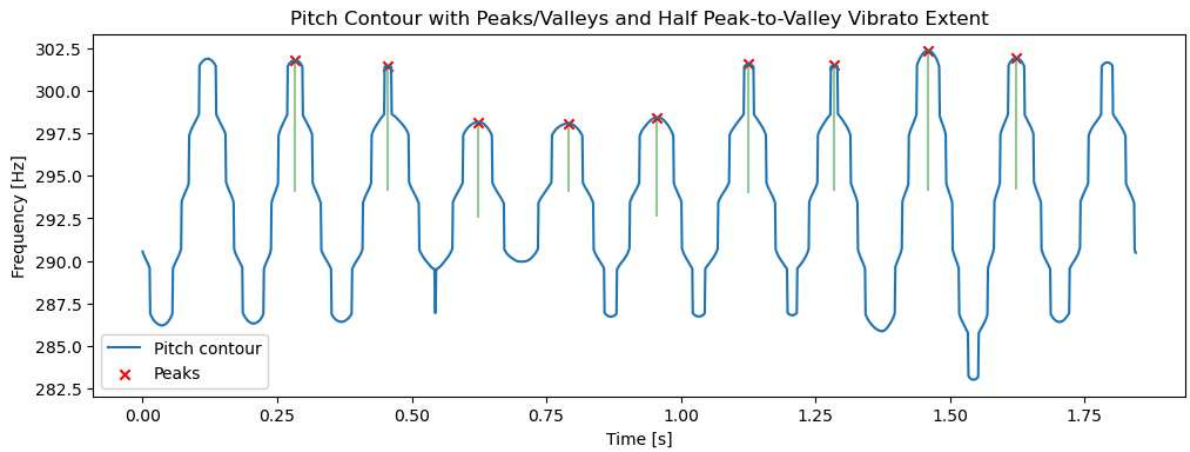
In [26]: vibExtent_cents, peaks, half_prominences = compute_vibrato_extent_cents(pitch_contour)
print(f"Estimated vibrato extent: {vibExtent_cents:.4f} (cents)")
# --- Visualization ---
plt.figure(figsize=(12,4))
plt.plot(t_pitch, pitch_contour, label='Pitch contour')
plt.scatter(t_pitch[peaks[1:-1]], pitch_contour[peaks[1:-1]], color='red', marker='o')
# plt.scatter(t_pitch[valleys], pitch_contour[valleys], color='green', marker='o', label='Valleys')

# Draw half peak-to-valley lines
for p, prom in zip(peaks[1:-1], half_prominences[1:-1]):
    plt.vlines(t_pitch[p], pitch_contour[p]-prom, pitch_contour[p], color='green',
               label=f'Half peak-to-valley extent ({prom:.2f} cents)')

plt.xlabel('Time [s]')
plt.ylabel('Frequency [Hz]')
plt.title('Pitch Contour with Peaks/Valleys and Half Peak-to-Valley Vibrato Extent')
plt.legend()
plt.show()

```

Estimated vibrato extent: 39.8193 (cents)



Now that we have a synthetic signal we have more than one harmonic to analyze. Let's look at the amplitude envelope of the first and second harmonic:

```
In [27]: # Filter around the fundamental and calculate hilbert envelope
meanFreq = np.mean(pitch_contour)

# --- Compute envelopes ---
envelope1, filtered1, f_s = returnEnvelope(signal50, f_s, meanFreq)
envelope2, filtered2, f_s = returnEnvelope(signal50, f_s, 2 * meanFreq)

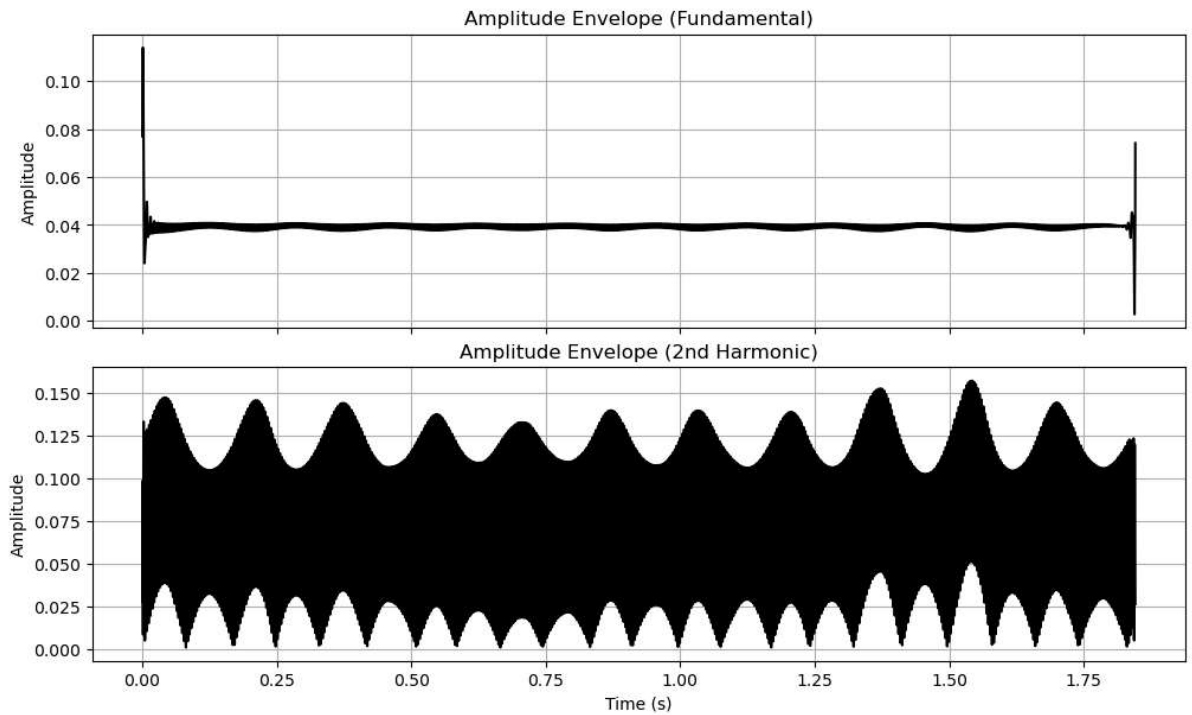
# --- Plot ---
fig, axs = plt.subplots(
    2, 1,
    figsize=(10, 6),
    sharex=True,
    constrained_layout=True
)

# Fundamental envelope
axs[0].plot(t50, envelope1, color='black')
axs[0].set_title('Amplitude Envelope (Fundamental)')
axs[0].set_ylabel('Amplitude')
axs[0].grid(True)

# 2nd harmonic envelope
axs[1].plot(t50, envelope2, color='black')
axs[1].set_title('Amplitude Envelope (2nd Harmonic)')
axs[1].set_xlabel('Time (s)')
axs[1].set_ylabel('Amplitude')
axs[1].grid(True)

# Optional: zoom in time
# axs[0].set_xlim(0.5, 1.5)

plt.show()
```

```
In [28]: def visualize_autocorrVib3HzLocal(pitch_contour, f_s_contour):
    x = pitch_contour
    n = len(x)

    # --- Autocorrelation ---
    acorr = sm.tsa.acf(x, nlags=n-1)

    # 95% confidence interval
    highCI = 1.96 / math.sqrt(n)
    lowCI = -highCI

    # Frequency → lag conversion
    frame10Hz = math.floor(f_s_contour / 10)
    frame3Hz = math.floor(f_s_contour / 3)

    # Segment of interest
    acorr_segment = acorr[frame10Hz:frame3Hz]
    peaks, _ = find_peaks(acorr_segment)

    vibratoFreq = np.nan
    best_lag = None

    if len(peaks) > 0:
        peak_values = acorr_segment[peaks]
        valid_indices = np.where(peak_values > highCI)[0]

        if len(valid_indices) > 0:
            best_peak_idx = valid_indices[np.argmax(peak_values[valid_indices])]
            best_lag = peaks[best_peak_idx] + frame10Hz
            vibratoFreq = f_s_contour / best_lag

    # --- Visualization ---
    lags = np.arange(len(acorr))
    freqs = f_s_contour / np.maximum(lags, 1) # avoid div-by-zero

    plt.figure(figsize=(12, 5))
    plt.plot(freqs, acorr, label='Autocorrelation', color='black')

    # Highlight search band
    plt.axvspan(3, 10, color='gray', alpha=0.2, label='Search range (3-10 Hz)')
```

```

# Confidence interval
plt.axhline(highCI, color='red', linestyle='--', label='95% CI')
plt.axhline(lowCI, color='red', linestyle='--')

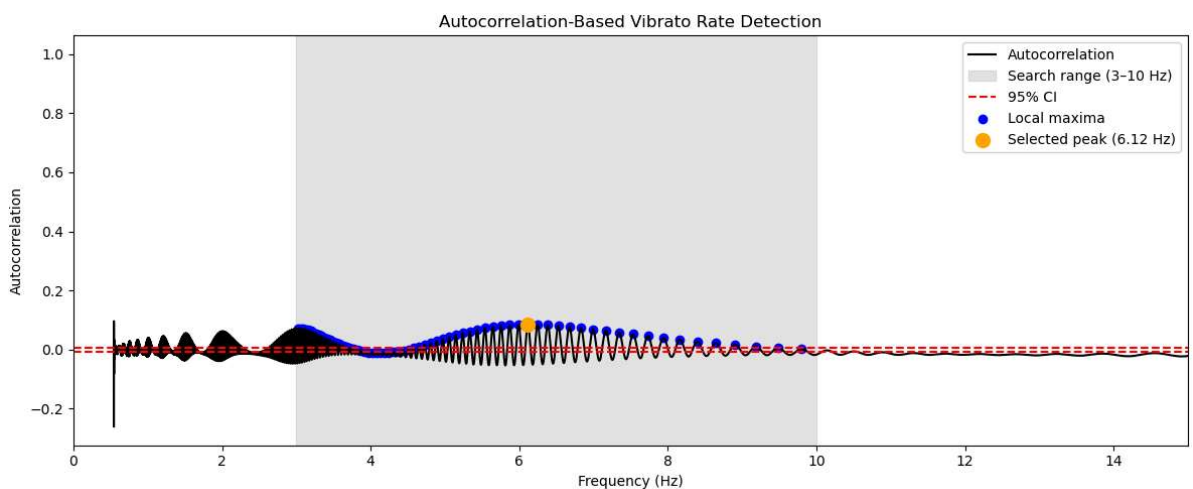
# Plot peaks
if len(peaks) > 0:
    peak_lags = peaks + frame10Hz
    peak_freqs = f_s_contour / peak_lags
    plt.scatter(
        peak_freqs,
        acorr[peak_lags],
        color='blue',
        label='Local maxima'
    )

# Highlight chosen peak
if best_lag is not None:
    best_freq = f_s_contour / best_lag
    plt.scatter(
        best_freq,
        acorr[best_lag],
        color='orange',
        s=100,
        zorder=5,
        label=f'Selected peak ({best_freq:.2f} Hz)'
    )

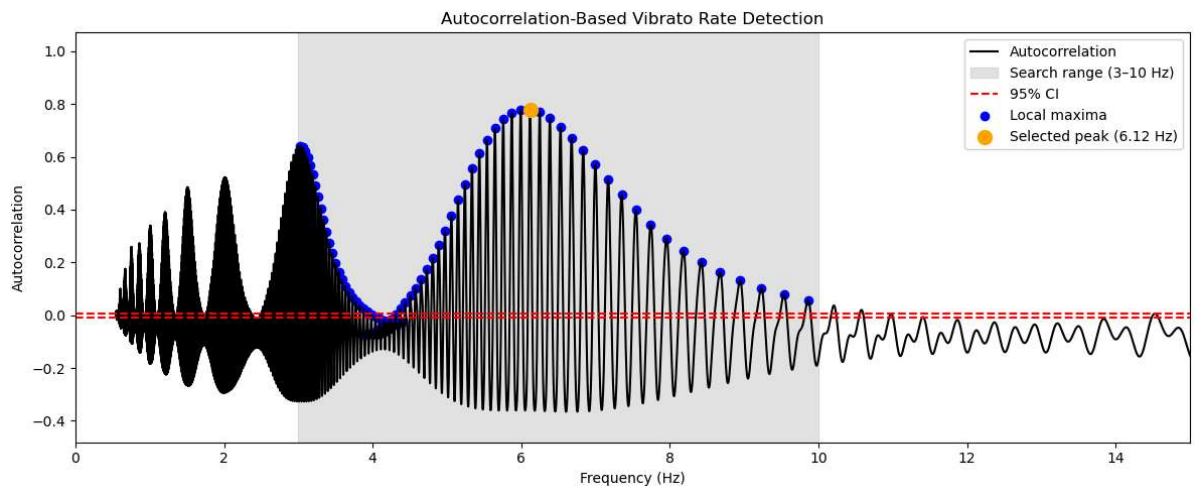
plt.xlim(0, 15)
plt.xlabel('Frequency (Hz)')
plt.ylabel('Autocorrelation')
plt.title('Autocorrelation-Based Vibrato Rate Detection')
plt.legend()
plt.tight_layout()
plt.show()

return vibratoFreq
vib_rate_est = visualize_autocorrVib3HzLocal(envelope1, f_s)
print(f"Estimated vibrato rate: {vib_rate_est:.2f} Hz")
vib_rate_est = visualize_autocorrVib3HzLocal(envelope2, f_s)
print(f"Estimated vibrato rate: {vib_rate_est:.2f} Hz")

```



Estimated vibrato rate: 6.12 Hz



Estimated vibrato rate: 6.12 Hz

There is some harmonic behavior happening in the envelope

```
In [29]: def visualize_envelope_fft(envelope, fs, max_freq=20, Type=""):
    """
    Visualize FFT of an envelope.

    Parameters
    -----
    envelope : np.ndarray
        Amplitude envelope (e.g., abs(hilbert(signal)))
    fs : float
        Sampling rate of the envelope (Hz)
    max_freq : float
        Maximum frequency to display (Hz)
    """

    # Remove DC offset (VERY important for envelope FFT)
    env = envelope - np.mean(envelope)

    # FFT
    N = len(env)
    fft_vals = np.fft.rfft(env)
    freqs = np.fft.rfftfreq(N, d=1/fs)

    magnitude = np.abs(fft_vals)

    # Plot
    plt.figure(figsize=(8, 4))
    plt.plot(freqs, magnitude, color='black')
    band_mask = (freqs >= 3) & (freqs <= 10)

    if not np.any(band_mask):
        return np.nan

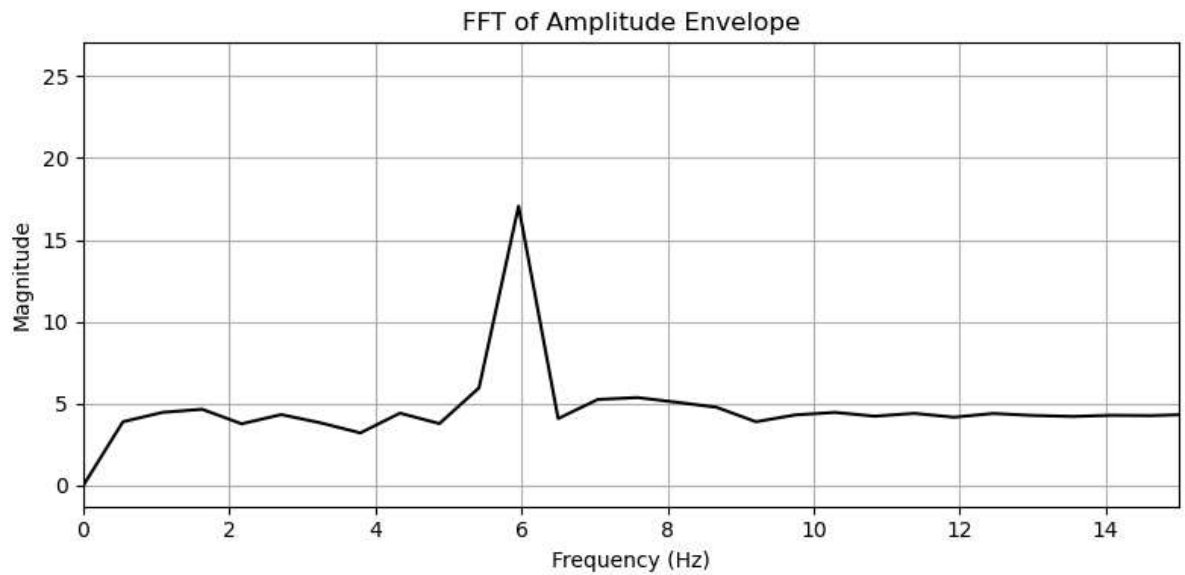
    band_mag = magnitude[band_mask]
    band_freqs = freqs[band_mask]

    # Peak frequency
    peak_freq = band_freqs[np.argmax(band_mag)]

    plt.xlim(0, max_freq)
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.title(f'FFT of {Type}')
    plt.grid(True)
    plt.tight_layout()
```

```
plt.show()

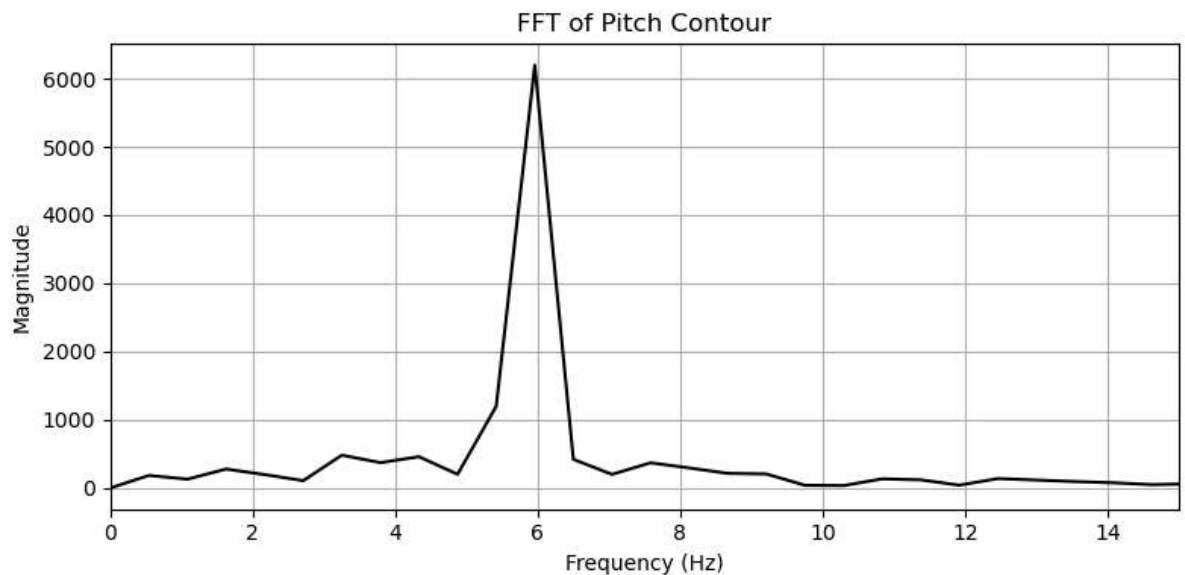
return peak_freq
peakFreq_am = visualize_envelope_fft(envelope1, f_s, max_freq=15, Type="Amplitude Envelope")
print(f"Dominant Vibrato Frequency (AM): {peakFreq_am:.2f} HZ")
```



Dominant Vibrato Frequency (AM): 5.96 HZ

That's much cleaner. It works with the pitch contour as well:

```
In [30]: peakFreq_fm = visualize_envelope_fft(pitch_contour, f_s_contour, max_freq=15, Type="Pitch Contour")
print(f"Dominant Vibrato Frequency (FM): {peakFreq_fm:.2f} HZ")
```



Dominant Vibrato Frequency (FM): 5.96 HZ

```
In [31]: fig, axes = plt.subplots(
    2, 1,
    figsize=(12, 7),
    sharex=True,
    constrained_layout=True
)

# -----
# Fundamental (envelope1)
# -----
extent_f0, peaks_f0, prominences_f0 = compute_vibrato_extent_hilbert(
```

```

        envelope1, f_s, peakFreq_am
    )
    half_prom_f0 = prominences_f0 / 2

    axs[0].plot(t50, envelope1, color='orange', label='Hilbert envelope (f0)')
    axs[0].scatter(
        t50[peaks_f0], envelope1[peaks_f0],
        color='red', marker='x', label='Detected peaks'
    )

    for p, prom in zip(peaks_f0, half_prom_f0):
        axs[0].vlines(
            t50[p],
            envelope1[p] - prom,
            envelope1[p],
            color='green',
            alpha=0.4
        )

    vals = envelope1[peaks_f0[1]:peaks_f0[-1]]
    ymin, ymax = vals.min(), vals.max()
    yrange = ymax - ymin
    axs[0].set_ylim(ymin - 0.1 * yrange, ymax + 0.1 * yrange)

    axs[0].set_title(
        f'Fundamental (Estimated extent [percent mean amplitude] = {extent_f0:.3f})'
    )
    axs[0].set_ylabel('Amplitude')
    axs[0].legend()
    axs[0].grid(True)

    # -----
    # Second harmonic (envelope2)
    # -----
    extent_2f0, peaks_2f0, prominences_2f0 = compute_vibrato_extent_hilbert(
        envelope2, f_s, peakFreq_am
    )
    half_prom_2f0 = prominences_2f0 / 2

    axs[1].plot(t50, envelope2, color='orange', label='Hilbert envelope (2f0)')
    axs[1].scatter(
        t50[peaks_2f0], envelope2[peaks_2f0],
        color='red', marker='x', label='Detected peaks'
    )

    for p, prom in zip(peaks_2f0, half_prom_2f0):
        axs[1].vlines(
            t50[p],
            envelope2[p] - prom,
            envelope2[p],
            color='green',
            alpha=0.4
        )

    vals = envelope2[peaks_2f0[1]:peaks_2f0[-1]]
    ymin, ymax = vals.min(), vals.max()
    yrange = ymax - ymin
    axs[1].set_ylim(ymin - 0.1 * yrange, ymax + 0.1 * yrange)

    axs[1].set_title(
        f'Second Harmonic (Estimated extent [percent mean amplitude] = {extent_2f0:.3f})'
    )
    axs[1].set_xlabel('Time [s]')
    axs[1].set_ylabel('Amplitude')

```

```

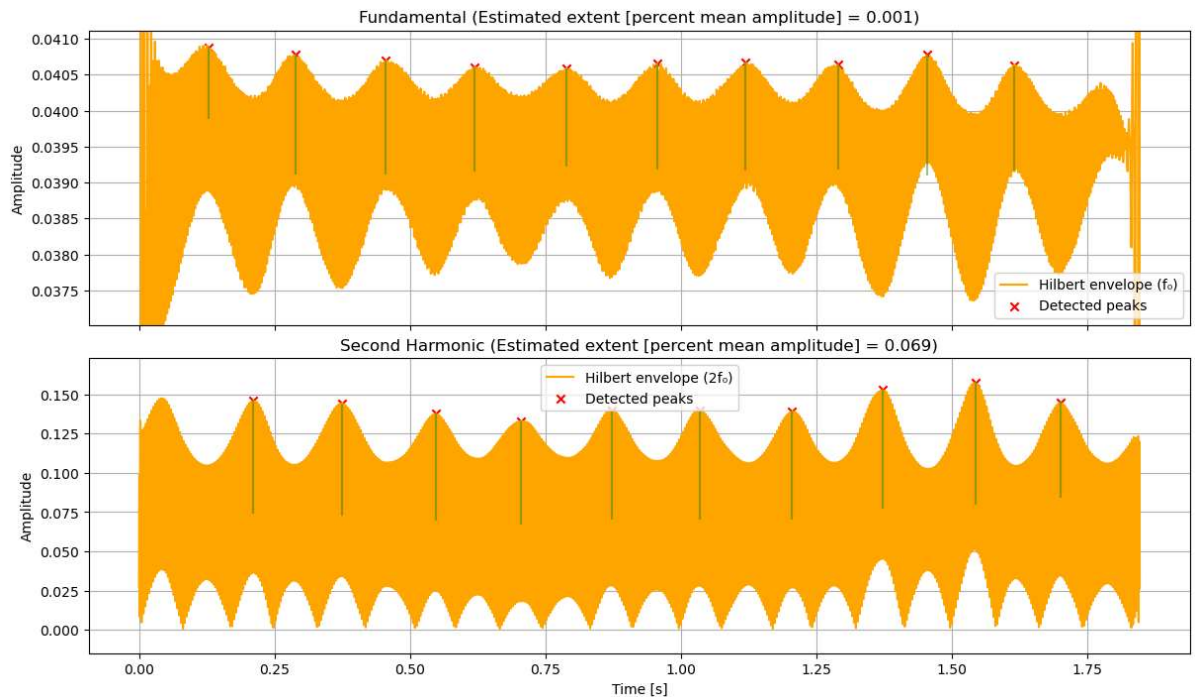
axs[1].legend()
axs[1].grid(True)

```

```

plt.show()

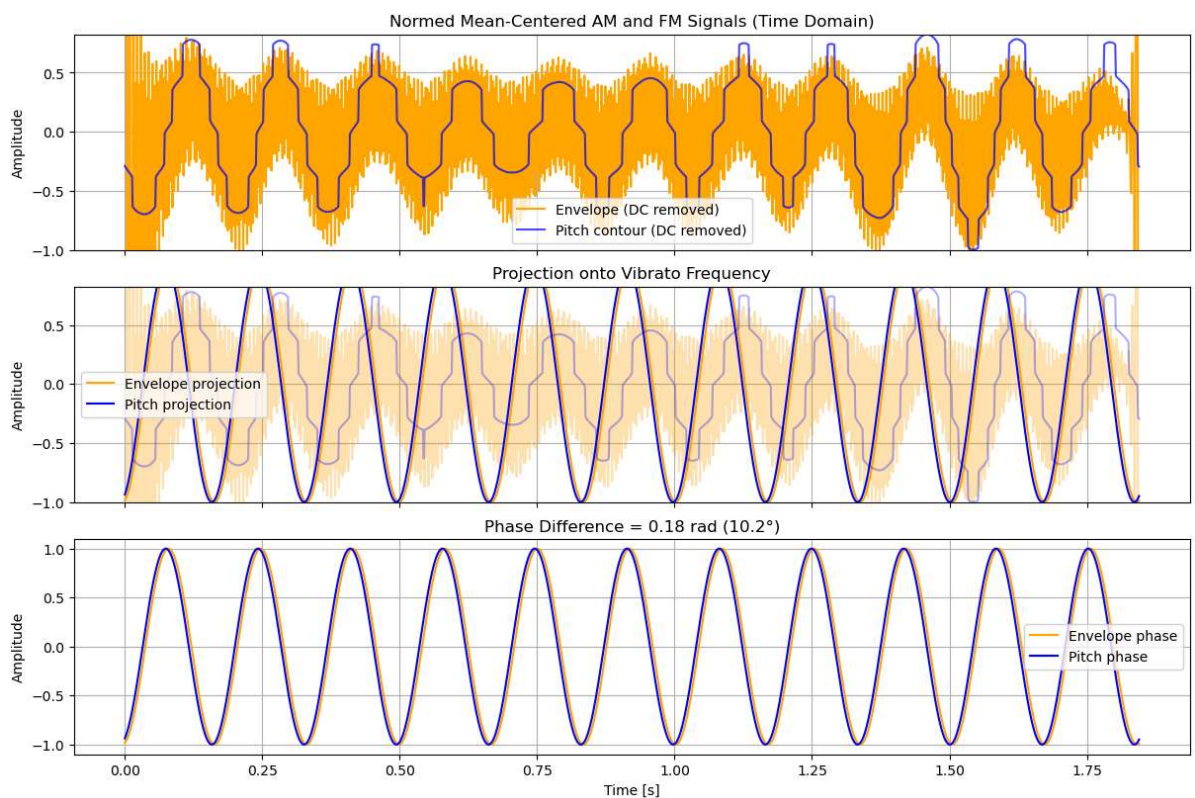
```



```

In [64]: phase_diff = visualize_phase_difference(
    envelope1,
    pitch_contour,
    fs_env=f_s,
    fs_pitch=f_s_contour,
    vibRate_f0=peakFreq_fm,
    vibRate_amp=peakFreq_am
)

```



```

In [65]: phase_diff = visualize_phase_difference(
    envelope2,

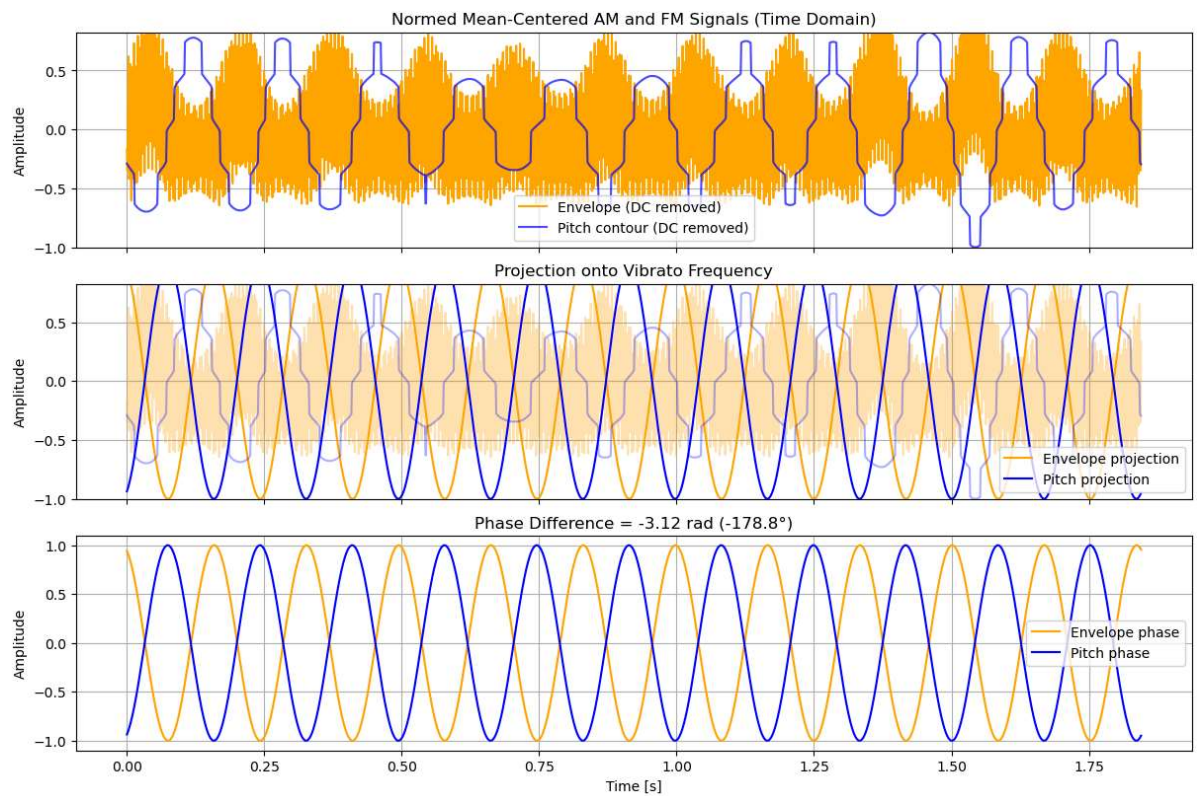
```



```

pitch_contour,
fs_env=f_s,
fs_pitch=f_s_contour,
vibRate_f0=peakFreq_fm,
vibRate_amp=peakFreq_am
)

```

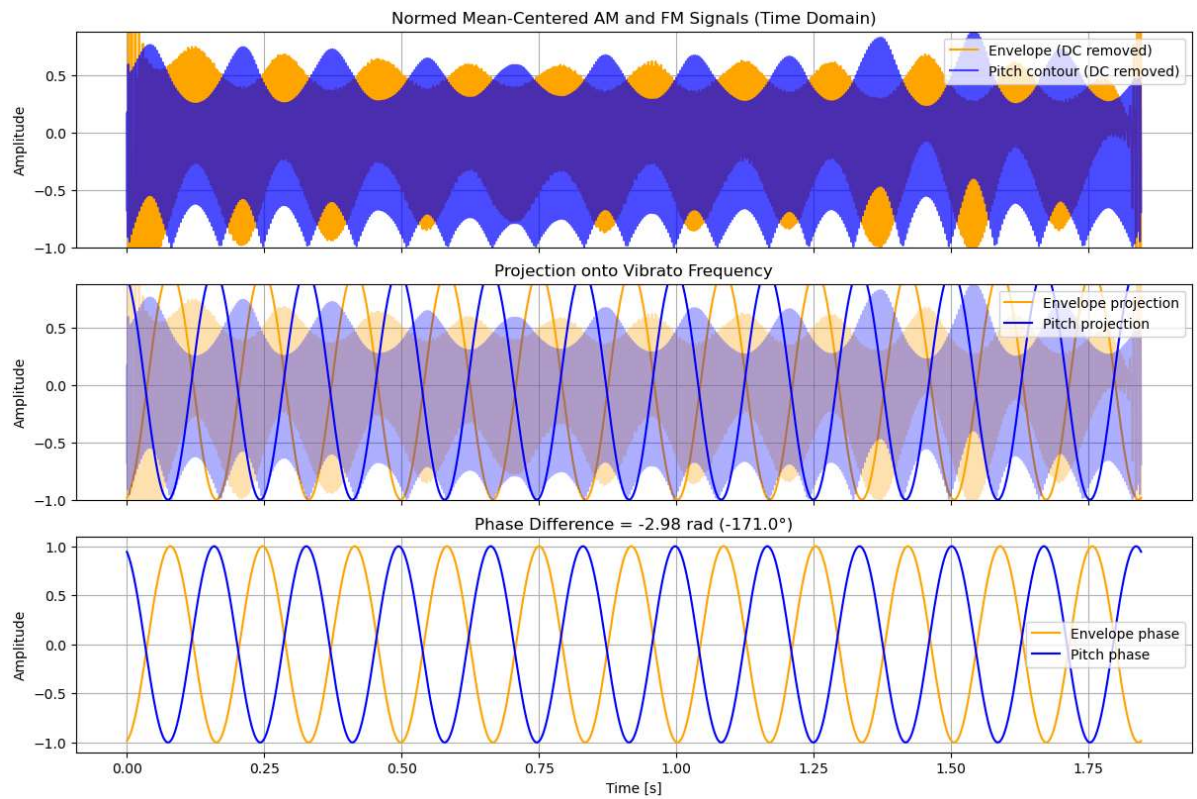


What!? That's wild. So envelope1 and envelope2 are $\pi/2$ phase delayed?

```

In [66]: phase_diff = visualize_phase_difference(
    envelope1,
    envelope2,
    fs_env=f_s,
    fs_pitch=f_s,
    vibRate_f0=peakFreq_am,
    vibRate_amp=peakFreq_am
)

```

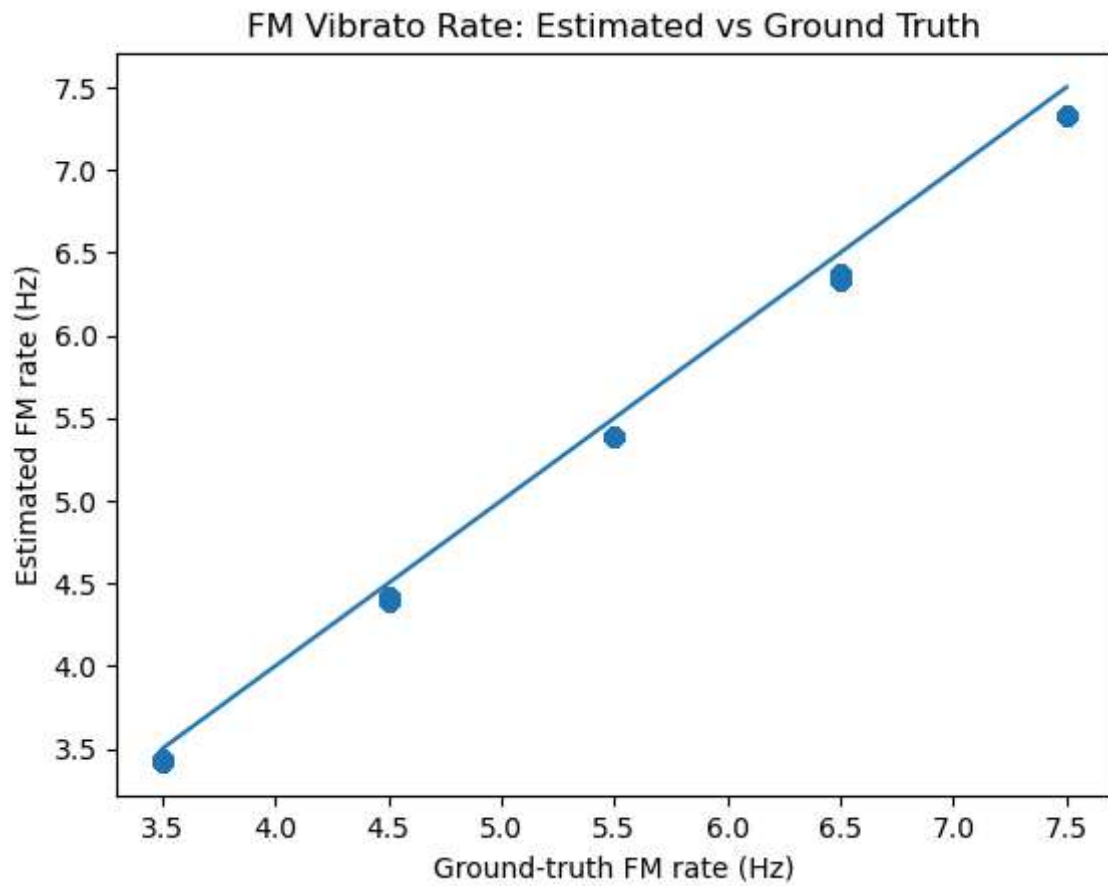



Ok, now to visualize a parameter sweep:

```
In [56]: import itertools
import pandas as pd
# --- Sweep ranges ---
f0s = [294, 659]
vib_rates = np.arange(3.0, 8.5, 1) # Hz
vib_extents = np.arange(15.0, 80.0, 20.0) # cents (or Hz if that's your definition)
am_depth_percent = np.linspace(0.005, 0.025, 5)
phase_offsets = np.array([0, 0.5*np.pi, np.pi, 1.5*np.pi])
df = pd.read_pickle("parameterSweep.pkl")
```

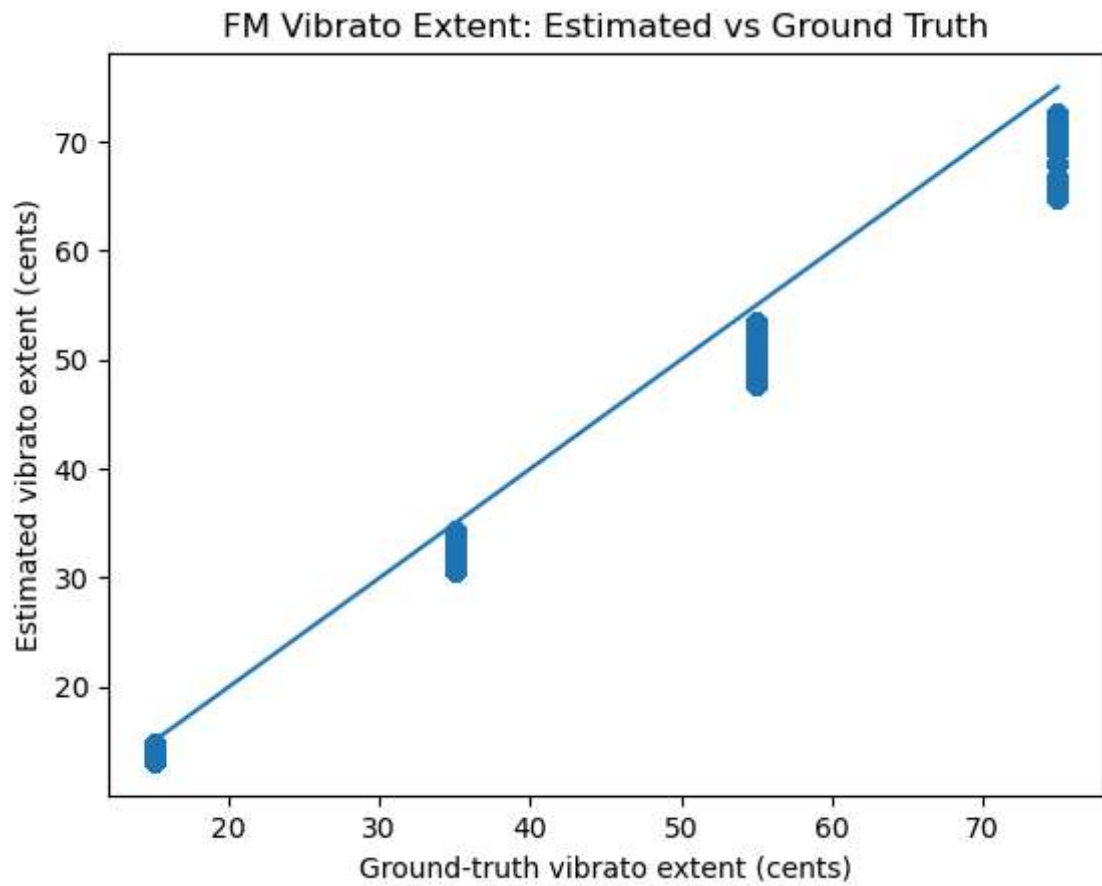
```
In [99]: mask = df['condition'].str.contains('FM')

plt.figure()
plt.scatter(
    df.loc[mask, 'vibRate_fm_gt'],
    df.loc[mask, 'vibRate_fm'],
    alpha=0.6
)
lims = [df['vibRate_fm_gt'].min(), df['vibRate_fm'].max()]
plt.plot(lims, lims)
plt.xlabel('Ground-truth FM rate (Hz)')
plt.ylabel('Estimated FM rate (Hz)')
plt.title('FM Vibrato Rate: Estimated vs Ground Truth')
# plt.savefig('fm_rate_est_vs_gt.png', dpi=300)
plt.show()
plt.close()
```



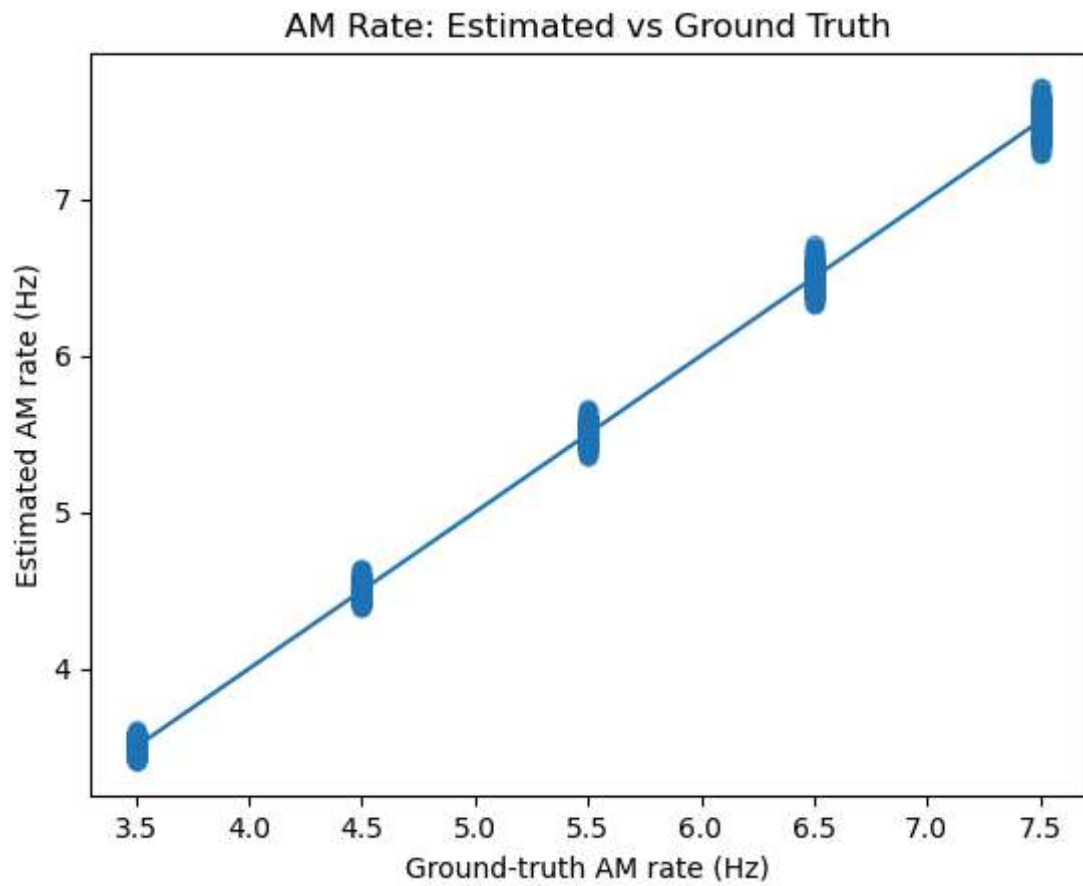
In [100...

```
plt.figure()
plt.scatter(
    df.loc[mask, 'vibExtent_fm_gt'],
    df.loc[mask, 'vibExtent_f0'],
    alpha=0.6
)
lims = [df['vibExtent_fm_gt'].min(), df['vibExtent_fm_gt'].max()]
plt.plot(lims, lims)
plt.xlabel('Ground-truth vibrato extent (cents)')
plt.ylabel('Estimated vibrato extent (cents)')
plt.title('FM Vibrato Extent: Estimated vs Ground Truth')
# plt.savefig('fm_extent_est_vs_gt.png', dpi=300)
plt.show()
plt.close()
```



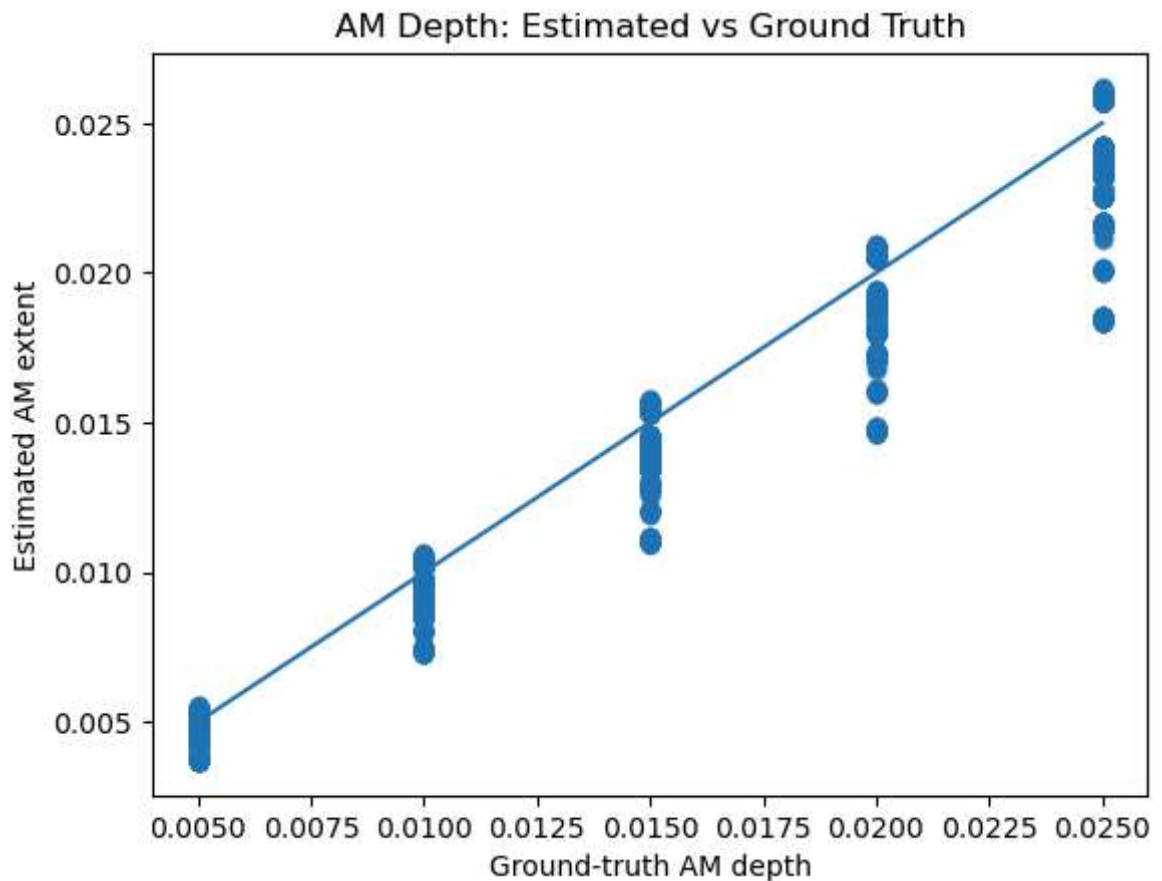
In [101...

```
plt.figure()
plt.scatter(
    df.loc[mask, 'vibRate_am_gt'],
    df.loc[mask, 'vibRate_amp'],
    alpha=0.6
)
lims = [df['vibRate_am_gt'].min(), df['vibRate_am_gt'].max()]
plt.plot(lims, lims)
plt.xlabel('Ground-truth AM rate (Hz)')
plt.ylabel('Estimated AM rate (Hz)')
plt.title('AM Rate: Estimated vs Ground Truth')
plt.show()
```



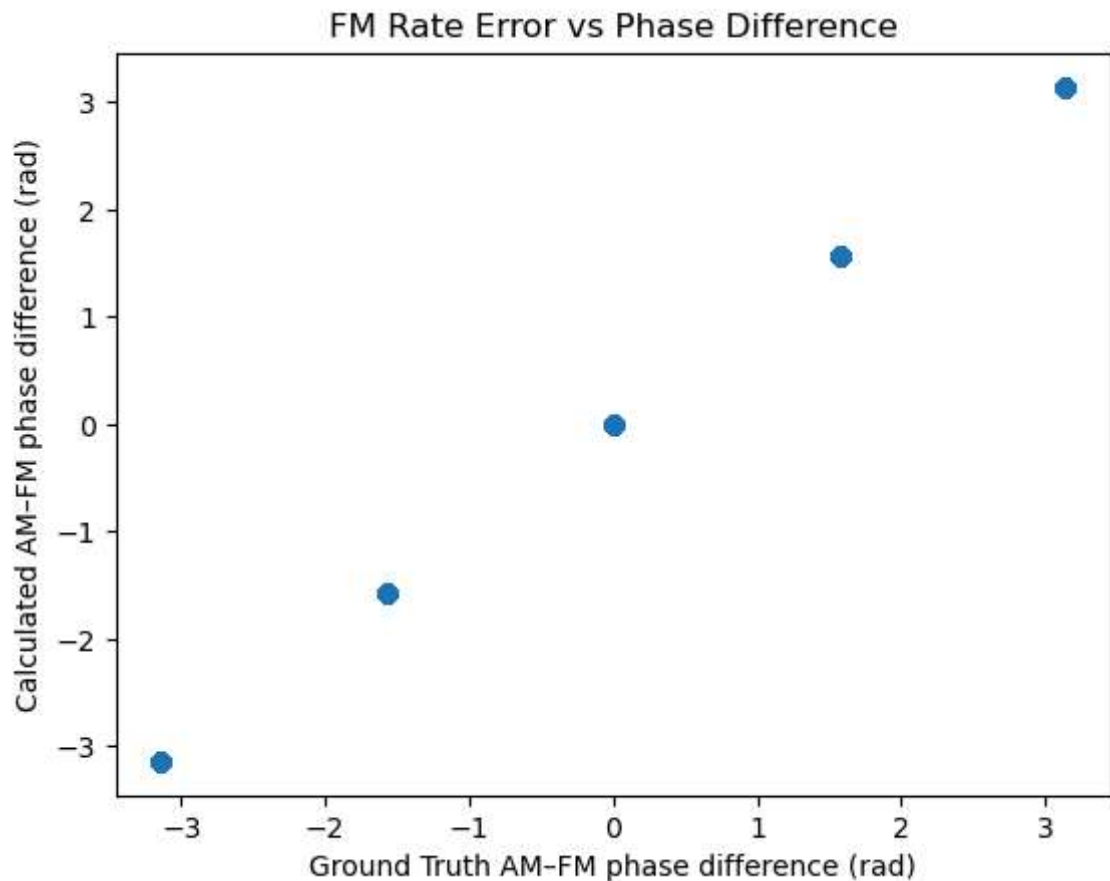
In [102...

```
plt.figure()
plt.scatter(
    df.loc[mask, 'amDepth_gt'],
    df.loc[mask, 'vibExtent_amp'],
    alpha=0.6
)
lims = [df['amDepth_gt'].min(), df['amDepth_gt'].max()]
plt.plot(lims, lims)
plt.xlabel('Ground-truth AM depth')
plt.ylabel('Estimated AM extent')
plt.title('AM Depth: Estimated vs Ground Truth')
# plt.savefig('am_extent_est_vs_gt.png', dpi=300)
plt.show()
```



```
In [61]: dfPhase = df[df['phaseDiff'] != False].copy() # Remove the signals that had different
# dfPhase['phaseDiff_gt'] = dfPhase['phaseDiff_gt'].apply(lambda x: (x + np.pi) % (
def flip2Pi(x):
    while np.abs(x) > np.pi:
        if x < - np.pi:
            x += 2*np.pi
        if x > np.pi:
            x -= 2*np.pi
    return x
dfPhase['phaseDiff_gt'] = dfPhase['phaseDiff_gt'].apply(lambda x: flip2Pi(x))
dfPhase['phaseDiff'] = dfPhase['phaseDiff_gt'].apply(lambda x: flip2Pi(x))
mask = dfPhase['condition'].str.contains('FM')
plt.figure()
plt.scatter(
    dfPhase.loc[mask, 'phaseDiff_gt'],
    dfPhase.loc[mask, 'phaseDiff'],
    alpha=0.6
)
# plt.axhline(0)
plt.xlabel('Ground Truth AM-FM phase difference (rad)')
plt.ylabel('Calculated AM-FM phase difference (rad)')
plt.title('FM Rate Error vs Phase Difference')

plt.show()
```



The parameters for the audio synthesized in Madde:

- vibRate_fm [4, 7] Hz
- vibExtent_fm [20, 60] Cents
- flutter filter f0 [4,7] Hz
- flutter amplitude percent [1, 2] %

Correlation plot:

```
In [58]: df.columns
```

```
Out[58]: Index(['f0', 'vibRate_fm_gt', 'vibExtent_fm_gt', 'vibRate_am_gt', 'amDepth_gt',
              'condition', 'fmPhase_gt', 'amPhase_gt', 'phaseDiff_gt', 'vibRate_f0',
              'vibExtent_f0', 'vibRate_amp', 'vibExtent_amp', 'vibRate_amp_harm',
              'vibExtent_harm', 'vibExtent_ampPerc', 'phaseDiff'],
              dtype='object')
```

```
In [62]: cols = [
            # Ground truth
            'vibRate_fm_gt',
            'vibExtent_fm_gt',
            'vibRate_am_gt',
            'amDepth_gt',
            'phaseDiff_gt',

            # Estimates
            'vibRate_f0',
            'vibExtent_f0',
            'vibRate_amp',
            'vibExtent_amp',
            'phaseDiff'
        ]
```

```

df_corr = df[cols].dropna()
df_corr['phaseDiff_gt'] = df_corr['phaseDiff_gt'].apply(lambda x: flip2Pi(x))
df_corr['phaseDiff'] = df_corr['phaseDiff_gt'].apply(lambda x: flip2Pi(x))
corr = df_corr.corr()

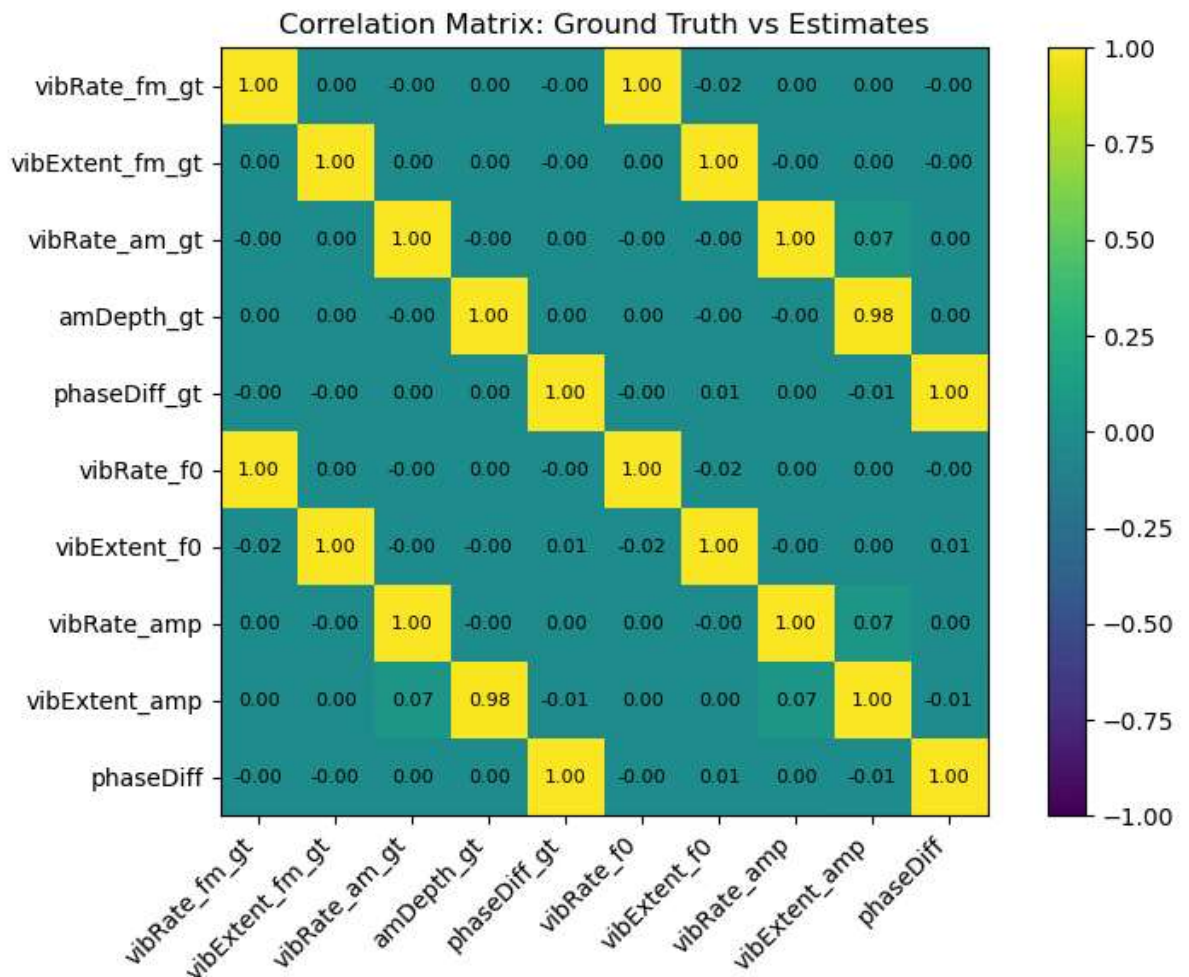
plt.figure(figsize=(8,6))
im = plt.imshow(corr, vmin=-1, vmax=1)
plt.colorbar(im, fraction=0.046)

plt.xticks(range(len(cols)), cols, rotation=45, ha='right')
plt.yticks(range(len(cols)), cols)
plt.title('Correlation Matrix: Ground Truth vs Estimates')

for i in range(len(cols)):
    for j in range(len(cols)):
        plt.text(j, i, f"{corr.iloc[i, j]:.2f}",
                 ha='center', va='center', fontsize=8)

plt.tight_layout()
plt.show()

```



```
In [37]: results_df = pd.read_pickle('maddeSweepAutocorr.pkl')
```

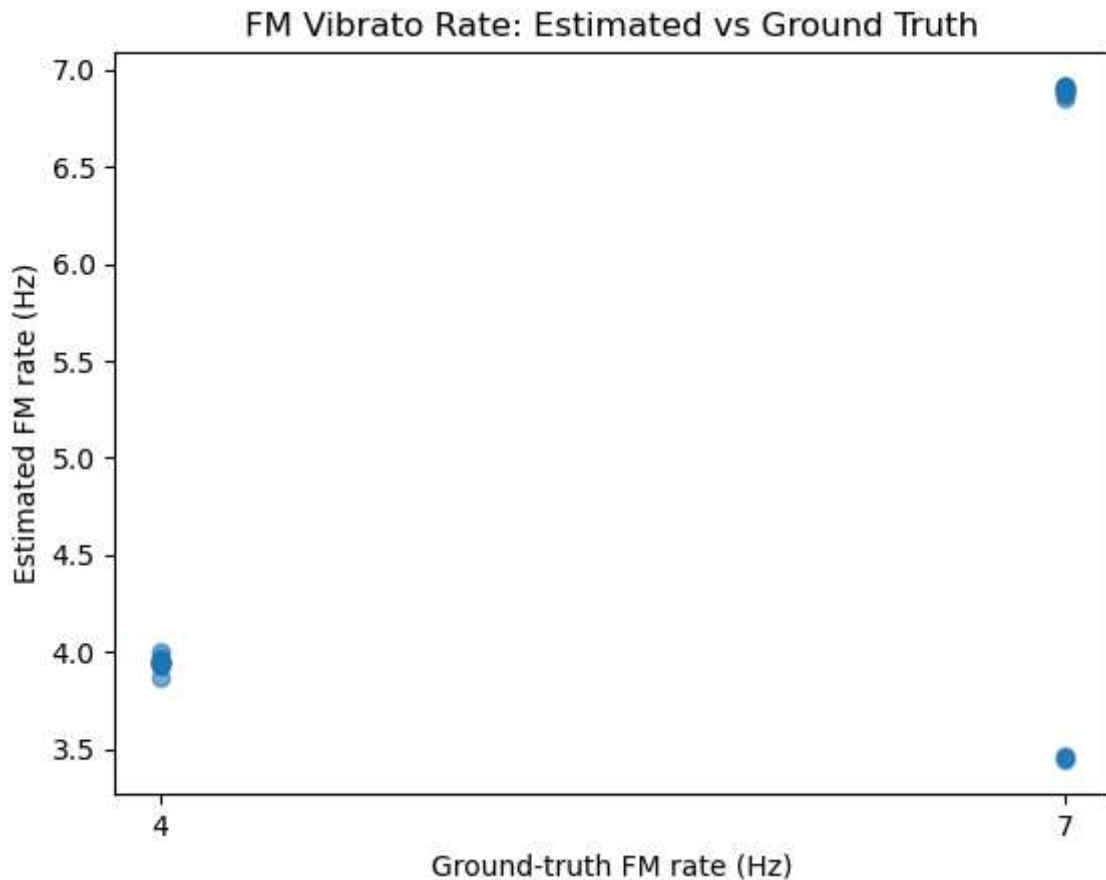
```

In [38]: # --- FM Vibrato Rate ---
plt.figure()
plt.scatter(
    results_df['vibRateFM_gt'],
    results_df['vibRate_f0'],
    alpha=0.6
)
lims = [results_df['vibRateFM_gt'].min(), results_df['vibRateFM_gt'].max()]
# plt.plot(lims, lims)

```



```
plt.xlabel('Ground-truth FM rate (Hz)')
plt.ylabel('Estimated FM rate (Hz)')
plt.title('FM Vibrato Rate: Estimated vs Ground Truth')
plt.savefig('fm_rate_est_vs_gt.png', dpi=300)
plt.show()
plt.close()
```



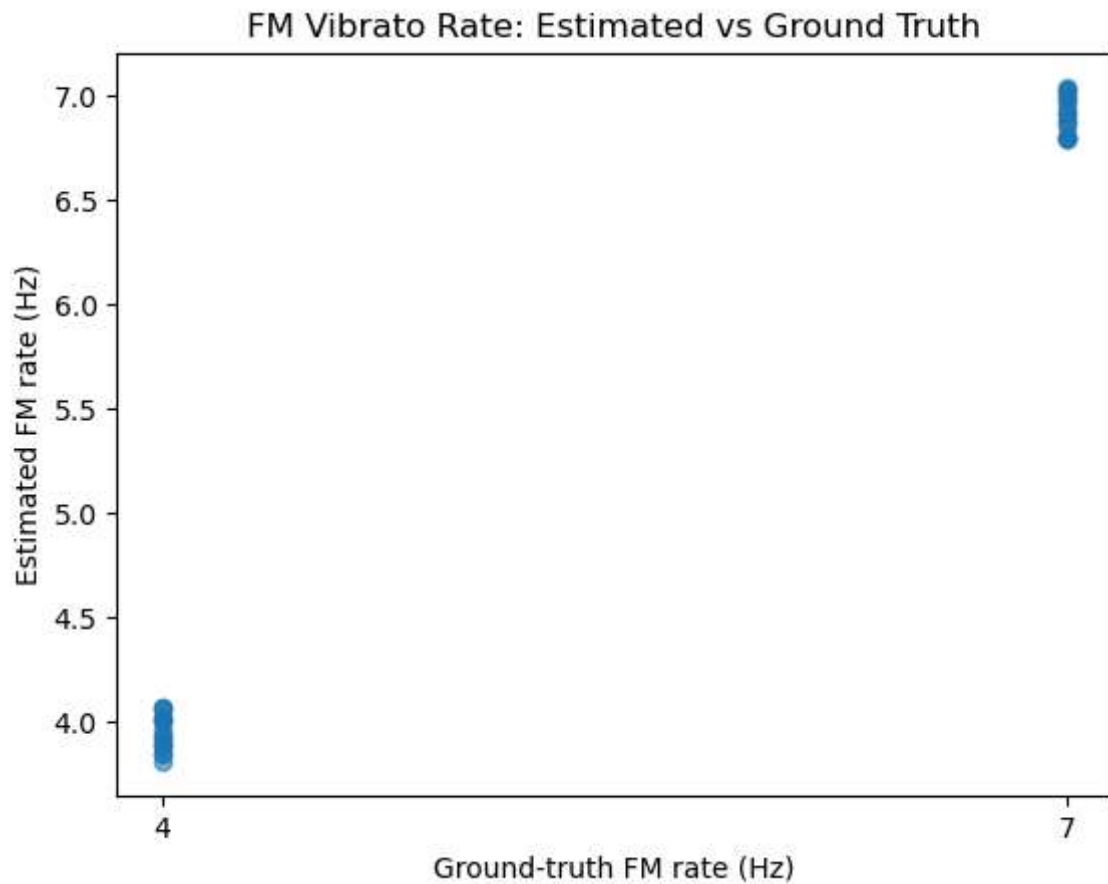
```
In [45]: results_df[((results_df['vibRateFM_gt'].astype('float') > 6) & (results_df['vibRateFM_est'] < 6))]
```

```
Out[45]: 8      294_7_20_4_1.wav
          9      294_7_20_4_2.wav
          25     659_7_20_4_2.wav
          Name: filename, dtype: object
```

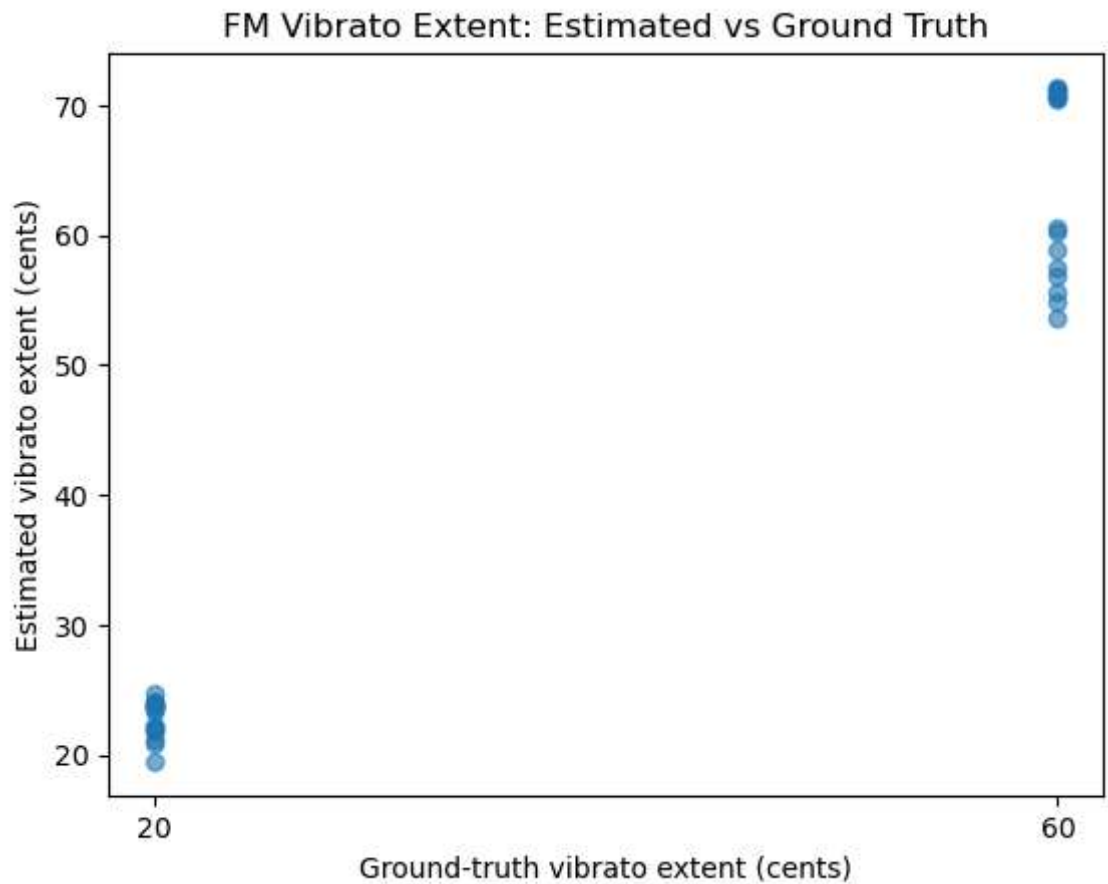
The three files with erroneous vibrato rate had low vibrato extent (20 cents) and were calculated with autocorrelation. Let's try the FFT method.

```
In [46]: results_df = pd.read_pickle('maddeSweepFFT.pkl')
```

```
In [47]: # --- FM Vibrato Rate ---
plt.figure()
plt.scatter(
    results_df['vibRateFM_gt'],
    results_df['vibRate_fm'],
    alpha=0.6
)
lims = [results_df['vibRateFM_gt'].min(), results_df['vibRateFM_gt'].max()]
# plt.plot(lims, lims)
plt.xlabel('Ground-truth FM rate (Hz)')
plt.ylabel('Estimated FM rate (Hz)')
plt.title('FM Vibrato Rate: Estimated vs Ground Truth')
plt.savefig('fm_rate_est_vs_gt.png', dpi=300)
plt.show()
plt.close()
```



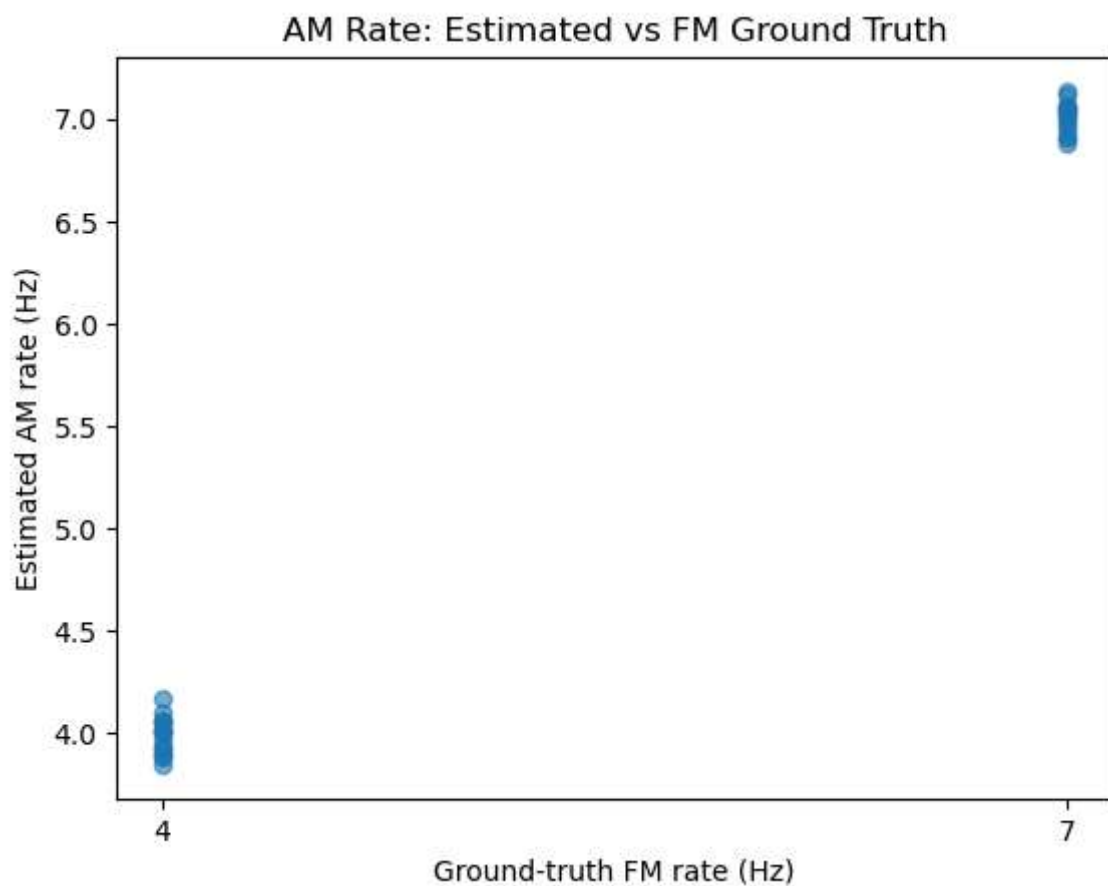
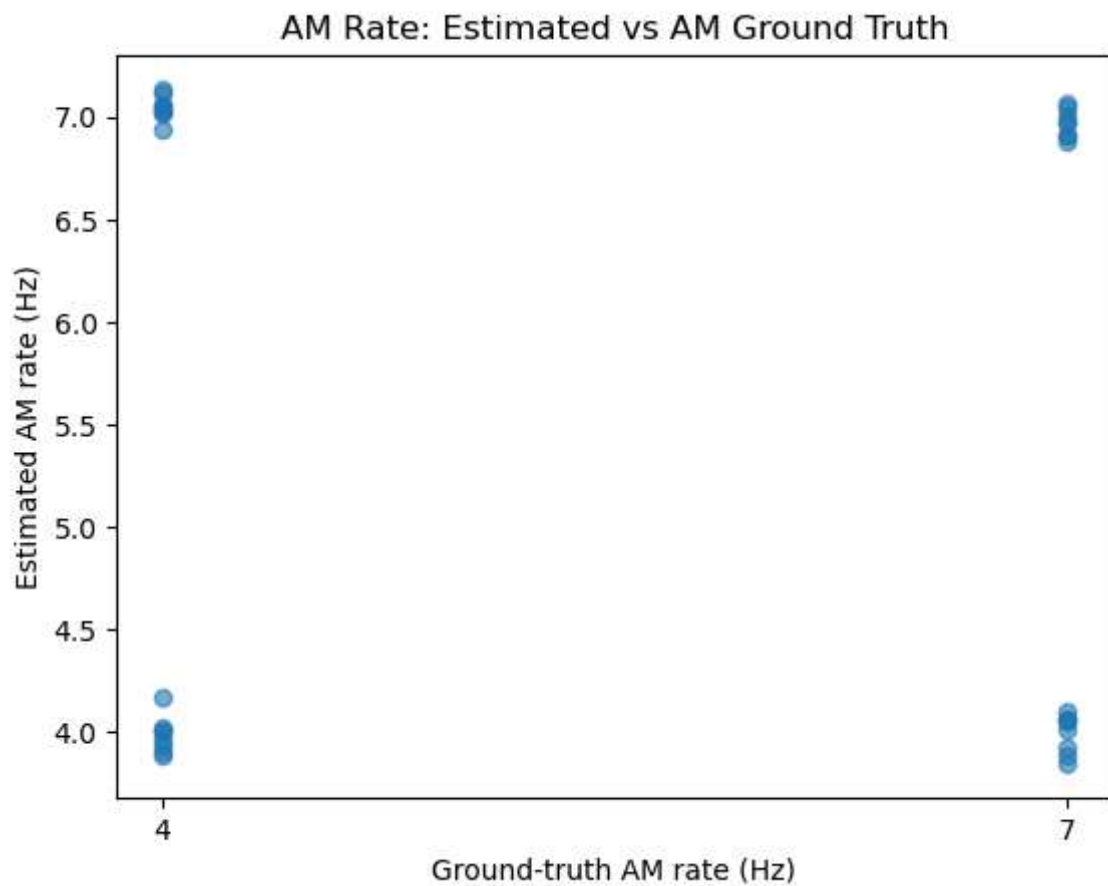
```
In [48]: # --- FM Vibrato Extent ---
plt.figure()
plt.scatter(
    results_df['vibExtentFM_gt'],
    results_df['vibExtent_f0'],
    alpha=0.6
)
lims = [results_df['vibExtentFM_gt'].min(), results_df['vibExtentFM_gt'].max()]
# plt.plot(lims, lims)
plt.xlabel('Ground-truth vibrato extent (cents)')
plt.ylabel('Estimated vibrato extent (cents)')
plt.title('FM Vibrato Extent: Estimated vs Ground Truth')
plt.savefig('fm_extent_est_vs_gt.png', dpi=300)
plt.show()
plt.close()
```



The AM Vibrato Rate is not really synthesized as such. It is a flutter (random) simulation. Our AM modulation is picking up the FM modulation.

```
In [51]: # --- AM Rate ---
plt.figure()
plt.scatter(
    results_df['vibRateAM_gt'],
    results_df['vibRate_amp'],
    alpha=0.6
)
lims = [results_df['vibRateAM_gt'].min(), results_df['vibRateAM_gt'].max()]
# plt.plot(lims, lims)
plt.xlabel('Ground-truth AM rate (Hz)')
plt.ylabel('Estimated AM rate (Hz)')
plt.title('AM Rate: Estimated vs AM Ground Truth')
plt.savefig('am_rate_est_vs_gt.png', dpi=300)
plt.show()
plt.close()

# --- AM Rate ---
plt.figure()
plt.scatter(
    results_df['vibRateFM_gt'],
    results_df['vibRate_amp'],
    alpha=0.6
)
# lims = [results_df['vibRateAM_gt'].min(), results_df['vibRateAM_gt'].max()]
# plt.plot(lims, lims)
plt.xlabel('Ground-truth FM rate (Hz)')
plt.ylabel('Estimated AM rate (Hz)')
plt.title('AM Rate: Estimated vs FM Ground Truth')
plt.savefig('am_rate_est_vs_FMgt.png', dpi=300)
plt.show()
plt.close()
```



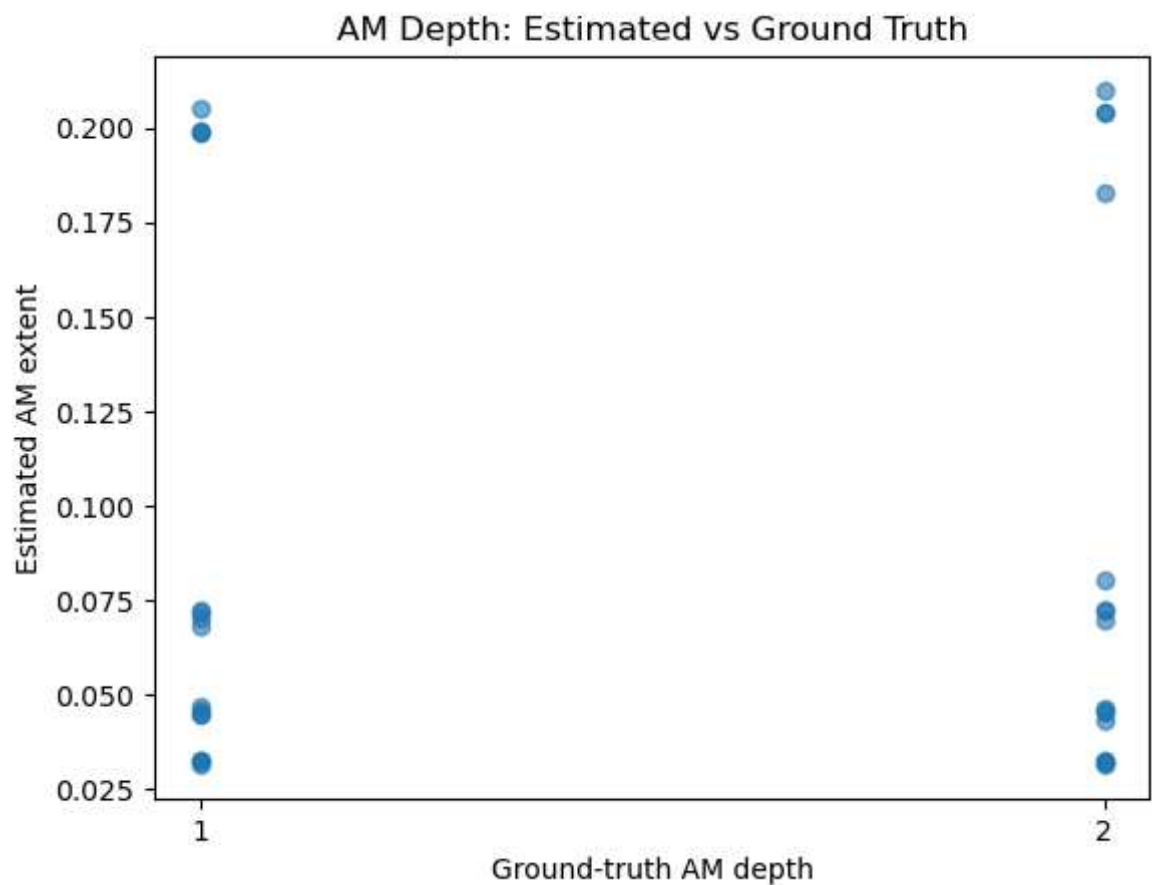
```
In [53]: # --- AM Depth / Extent ---
plt.figure()
plt.scatter(
    results_df['vibExtentAM_gt'],
    results_df['vibExtent_amp'],
    alpha=0.6
)
```

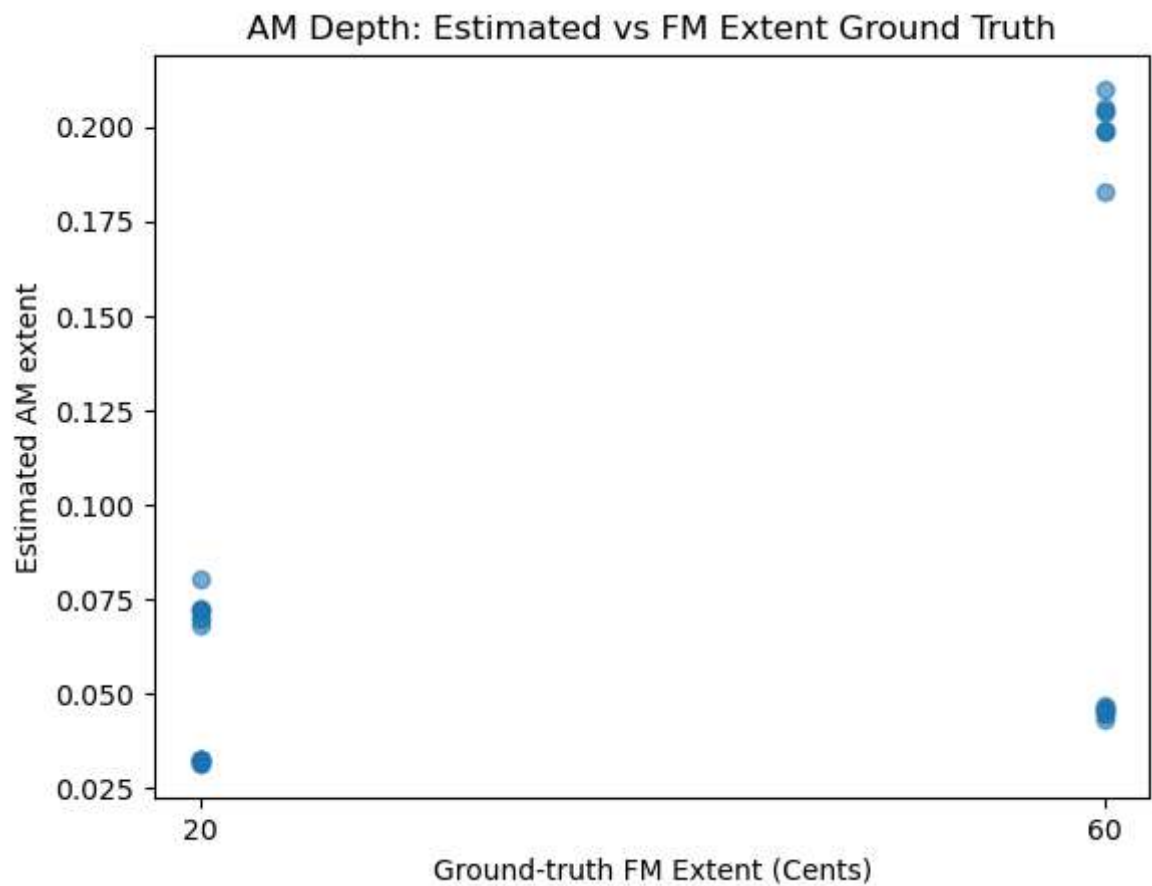
```

plt.xlabel('Ground-truth AM depth')
plt.ylabel('Estimated AM extent')
plt.title('AM Depth: Estimated vs Ground Truth')
plt.savefig('am_extent_est_vs_gt.png', dpi=300)
plt.show()
plt.close()

# --- AM Depth / Extent ---
plt.figure()
plt.scatter(
    results_df['vibExtentFM_gt'],
    results_df['vibExtent_amp'],
    alpha=0.6
)
plt.xlabel('Ground-truth FM Extent (Cents)')
plt.ylabel('Estimated AM extent')
plt.title('AM Depth: Estimated vs FM Extent Ground Truth')
plt.savefig('am_extent_est_vs_FMgt.png', dpi=300)
plt.show()
plt.close()

```





Correlation plot:

```
In [54]: cols = [
    # Ground truth
    'vibRateFM_gt',
    'vibExtentFM_gt',
    'vibRateAM_gt',
    'vibExtentAM_gt',
    # 'phaseDiff_gt',

    # Estimates
    'vibRate_f0',
    'vibExtent_f0',
    'vibRate_amp',
    'vibExtent_amp',
    # 'phaseDiff'
]

df_corr = results_df[cols].dropna()

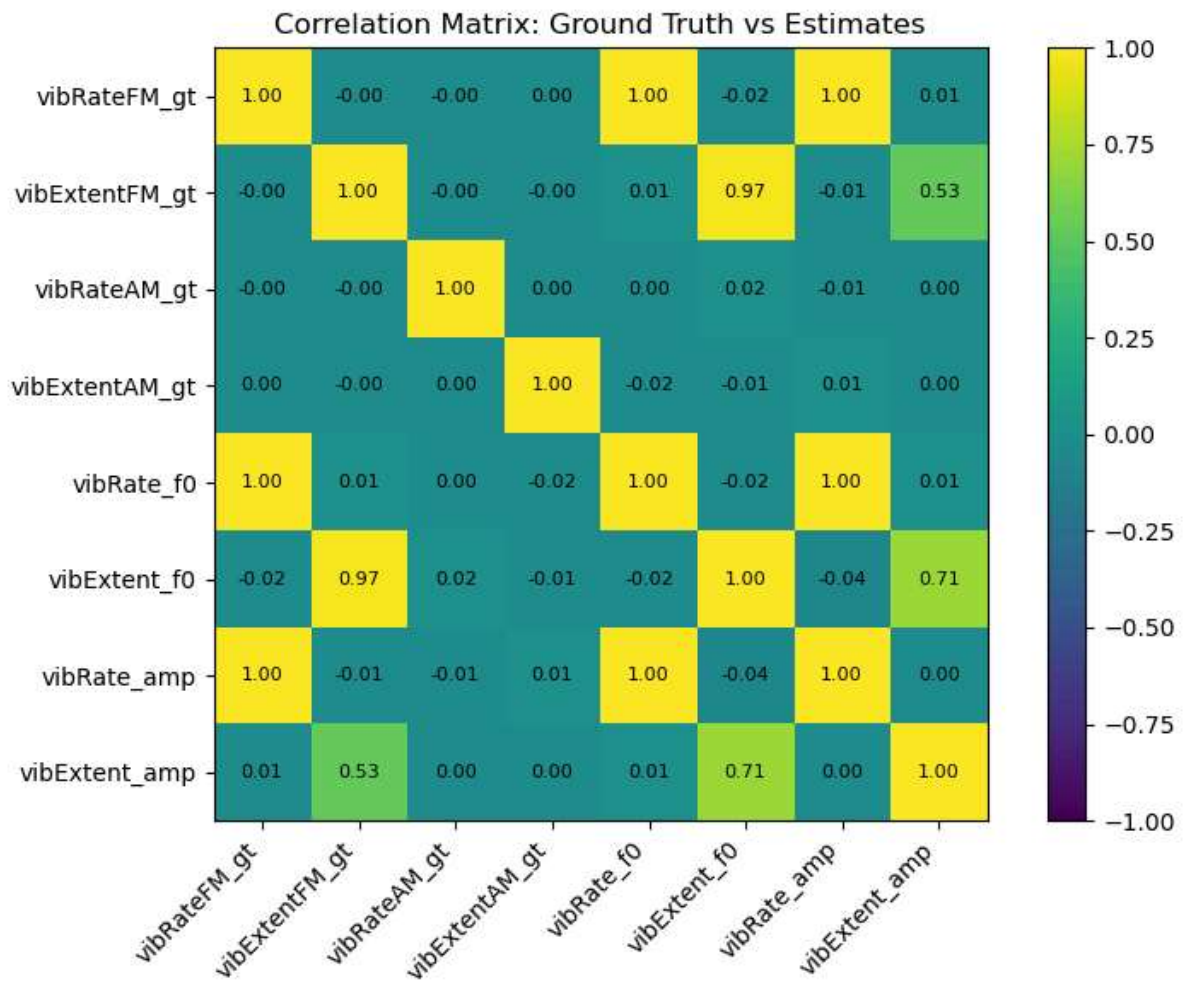
corr = df_corr.corr()

plt.figure(figsize=(8,6))
im = plt.imshow(corr, vmin=-1, vmax=1)
plt.colorbar(im, fraction=0.046)

plt.xticks(range(len(cols)), cols, rotation=45, ha='right')
plt.yticks(range(len(cols)), cols)
plt.title('Correlation Matrix: Ground Truth vs Estimates')

for i in range(len(cols)):
    for j in range(len(cols)):
        plt.text(j, i, f"{corr.iloc[i, j]:.2f}",
                 ha='center', va='center', fontsize=8)
```

```
plt.tight_layout()
plt.show()
```



In []: